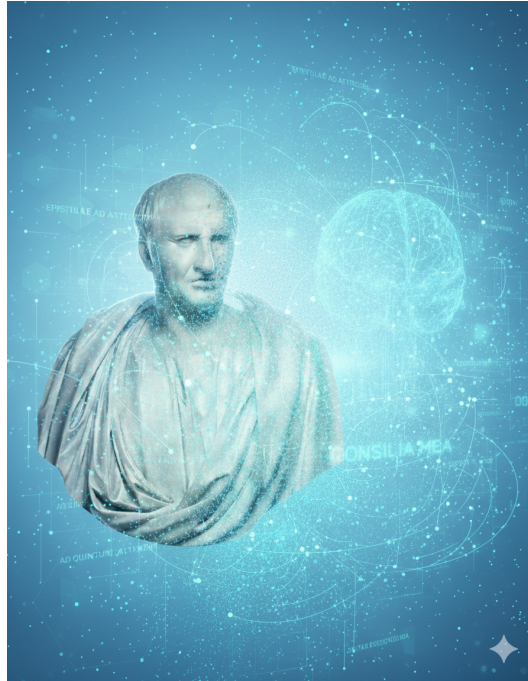


Phillip B. Ströbel & Felix K. Maier

Cicero Digital



Automatische Analyse eines antiken Briefwechsels

März 2026

ver. 2026.c

Inhaltsverzeichnis

1	Einführung	5
1.1	Cicero	6
1.2	Ciceros Briefe	7
1.3	Aufbau des Readers	9
2	Der Brief als Datenbasis	11
2.1	Briefe in der Antike	11
2.2	Die Datengrundlage bei Cicero	13
2.2.1	Perseus	14
2.2.2	Datenformat: XML	14
2.3	Erste Verarbeitungsschritte	23
2.3.1	Dateien finden	23
2.3.2	TEI und Namespace	23
2.3.3	Was ist ein Brief im XML?	24
2.3.4	Metadaten extrahieren	24
2.3.5	Absender und Empfänger als Heuristik	25
2.3.6	Vom XML zur Tabelle	25
2.3.7	Datenset exportieren	28
2.3.8	Erste statistische Checks	28
2.3.9	Visualisierung	29
2.3.10	Fazit	29
3	Diskurs und Pragmatik	31
3.1	Diskurs	32
3.2	Pragmatik	36
3.3	Textsegmentierung mit <i>spaCy</i>	36
3.3.1	Ziel dieses Abschnitts	36
3.3.2	Briefdaten einlesen	37
3.3.3	LatinCy initialisieren	37
3.3.4	Ein Brief als Beispiel	38
3.3.5	POS-Verteilungen (gesamt und nach Subkorpus)	39
3.3.6	Fazit	39
3.4	Diskurskohärenz	39
3.4.1	Rhetorical Structure Theory	40
3.4.2	Phatische Stabilisierung des Kontakts	41

3.4.3	Kohärenz als soziale Praxis	42
3.4.4	Elementary Discourse Units (EDUs)	43
3.4.5	EDU-Segmentierung für RST-Parsing	44
3.4.6	RST-Parsing (vereinfacht)	44
3.4.7	Digitale Diskursanalyse	45
3.5	Pragmatikannoation	48
4	Selektion und Lücken	49
4.1	Der ominöse Herausgeber?	49
4.2	Von TF-IDF zu Topic Modeling	51
4.3	TF-IDF	54
4.3.1	Ziel dieses Abschnitts	55
4.3.2	Daten laden	55
4.3.3	TF-IDF-Vektorisierung	55
4.3.4	Top-Wörter eines einzelnen Briefs	56
4.3.5	Die wichtigsten TF-IDF-Wörter im gesamten Korpus	57
4.3.6	Ähnliche Briefe mit Cosine Similarity	58
4.3.7	Visualisierung eines Similarity-Ausschnitts	58
4.3.8	Ein kleines Netzwerk ähnlicher Briefe	59
4.3.9	Fazit	60
4.4	Topic Modeling	60
4.4.1	Vorbereitung des Korpus	61
4.4.2	LDA-Modell mit gensim	61
4.4.3	Top-Wörter pro Topic anzeigen	62
4.4.4	Kohärenz des Modells prüfen	62
4.4.5	Dominantes Topic pro Brief	62
4.4.6	Verteilung der Topics visualisieren	64
4.4.7	Top-Wörter pro Topic als Balkendiagramm	64
4.4.8	Dokument-Topic-Matrix	65
4.4.9	Interaktive Visualisierung mit pyLDAvis	66
4.4.10	BERTopic	66
4.4.11	Fazit	67
	Litertaurverzeichnis	71

Session

1

Einführung

Lernziele Session 1: Einführung

Verstehen Du verstehst, wie digitale Methoden historische Fragestellungen an antiken Texten ermöglichen, strukturieren und zugleich begrenzen, und kannst deren Voraussetzungen und Annahmen benennen.

Analysieren Du kannst Ciceros Briefe sowohl als Texte als auch als Daten analysieren und zwischen *close reading* und skalierenden Verfahren reflektiert wechseln.

Anwenden Du kannst digitale Analyseverfahren (z.B. Metadaten, einfache Korpusanalysen, Netzwerke, Mustererkennung) gezielt einsetzen und mit historischer Kontextualisierung verbinden.

Bewerten Du kannst digitale Ergebnisse kritisch beurteilen, insbesondere im Hinblick auf Selektion, Überlieferung, Modellierungsentscheidungen und Interpretationsgrenzen.

Erzeugen Du kannst eine eigenständige, methodisch transparente Analyse zu Ciceros Briefen erarbeiten und in geeigneter Form präsentieren (z.B. als Wiki-Beitrag), inklusive nachvollziehbarer Fragestellung, Datenbasis und Interpretation.

Disclaimer: Dieser Reader hat nicht die Absicht, ein vollständiges historiographisches Porträt von Cicero nachzuzeichnen und wiederzugeben. Wir beschränken uns in Bezug auf die Nennung historischer Quellen und relevanter Forschungsbeiträge auf ein Minimum. Vielmehr geht es in diesem Reader darum, computerlinguistische Methoden auf den Briefwechsel anzuwenden. Daraus resultierende Ergebnisse werden nur vereinzelt in den Kanon der bereits bestehenden Forschung eingeordnet. Nichtsdestotrotz sind wir für Hinweise zu Fehlern sehr dankbar!

1.1 Cicero

Marcus Tullius Cicero (106–43 v. Chr.) war einer der bedeutendsten Persönlichkeiten in der Zeit der sich dem Ende nahenden Römischen Republik. Aber warum eigentlich? Jeder und jedem ist der Name eigentlich ein Begriff und durch seine zahlreichen überlieferten Werke ist Cicero als Person eigentlich gut fassbar. Trotzdem könnten die allermeisten wohl “nur” sagen, dass er ein berühmter Schriftsteller der lateinischen Klassik gewesen sei. Und das wissen wohl auch nur gerade diejenigen, welche in ihrer schulischen Laufbahn in den Genuss von Lateinunterricht gekommen sind (und lange genug dabei geblieben sind, bis der obligate Cicero-Text übersetzt werden musste).

Doch Cicero war weitaus mehr als bloss ein Schriftsteller. Schaut man sich einige seiner Schriften an, die bis in unsere Zeit überdauerten, wird schnell klar, dass Cicero in vielen Fachrichtungen unterwegs war. Viele seiner Reden (z. B. aus dem Jahre 66 v. Chr. mit dem Titel *De imperio Cn. Pompei*, “Für den Oberbefehl des Pompeius”) hielt er im Senat – als Politiker. Dafür finden wir auch andere Quellen, wie beispielsweise eine Inschrift aus der Hafenstadt Ostia, die verkündet, dass unter dem Konsulat Ciceros (63 v. Chr.) *muros* (“Mauern”) und *portas* (“Pforten”) errichtet wurden (siehe Abb. 1.1). Das Werk *Pro Archia* (“Für Archias”) stellt eine von Cicero im Jahre 63 v. Chr. gehaltene Verteidigungsrede dar, in welcher wir Cicero die Rolle als Anwalt für den Dichter Archias einnehmen sehen. Sein Text *De Divinatione* (“Über die Wahrsagung”), der kurz vor seinem Tod um 44 v. Chr. entstand, vermittelt uns Cicero als Philosophen. 55 v. Chr. entstand noch Ciceros *De oratore* (“Über den Redner”), aus welchem Cicero auch als Rhetoriker hervorgeht.

Und schliesslich gibt es als überlieferte Quellen über 900 Briefe (mehr Details im Unterkapitel 1.2 und generell im Kapitel 2). Hier schlüpft Cicero zwar auch in die im vorherigen Absatz erwähnten Rollen, aber wir erfahren noch viel mehr über Cicero als Person. Im Briefwechsel begegnet uns Cicero nämlich auch als Bruder des **Quintus Tullius Cicero** (102–43 v. Chr., in *Epistulae ad Quintum fratrem*, “Briefe an den Bruder Quintus”), Vater, Ehemann und guter Freund (in *Epistulae ad familiares*, “Briefe an seine Freunde”). Besonders ausführlich sind die Kontakte mit **Titus Pomponius Atticus** (110–32 v. Chr., *Epistulae ad Atticum*) und einem gewissen **Marcus Iunius Brutus** (85–42 v. Chr., *Epistulae ad M. Brutum*), der als Mitverschwörer gegen und Mitmörder des **Caesar** in die Geschichte einging. Damit hätten wir im Übrigen bereits die vier Teilkorpora des überlieferten Briefkorpus aufgezählt, mit denen wir uns in diesem Reader beschäftigen werden. Sie seien der besseren Übersicht wegen hier nochmals genannt:

- *ad Quintum fratrem*
- *ad familiares*
- *ad Atticum*
- *ad M. Brutum*



Abbildung 1.1 Inschrift aus Ostia, die von Cicero als Konsul (auf der Inschrift als Abkürzung COS wiedergegeben) zeugt (CIL XIV, 4707).

Transkription: *Senatus populusque romanus / coloniae ostiensium muros et portas fecit / M(arcus) Tullius Cicero co(n)s(ul) fecit locavitque / P(ublius) Clodius Pulcher tr(ibunus) p(opu)l(us) consummavit et probavit / portam vetustate corruptam [...] et [...] rv*

Übersetzung: Der Senat und das Römische Volk haben für die Kolonie der Ostiensier Mauern und Tore errichtet. Der Konsul Marcus Tullius Cicero hat (den Bau) veranlasst und vergeben. Der Volkstribun Publius Clodius Pulcher hat (ihn) vollendet und abgenommen. (Dies betraf) ein durch das Alter zerstörtes Tor ... und ...

1.2 Ciceros Briefe

Es liegt in der Natur des Mediums “Brief,” dass sich Cicero im Briefwechsel anders darstellt als in seinen übrigen Schriften: Der Umgangston kann, vor allem in Austausch mit seinen engsten Kontakten, schon als kolloquial bezeichnet werden (Manuwald, 2025, S. 797). Schon in Werken der Antike spricht Demetrios von Phaleron (360–280 v. Chr.), dem die Schrift *περὶ ἐρμηνείας* (“Über den Stil”) zugeordnet wird (was allerdings umstritten ist), in der Charakterisierung von Briefen davon, dass jeder Autor ein Abbild seiner Seele im Brief hinterlässt (siehe dazu auch Müller, 1980, der vom “Spiegel der Seele” spricht). Dieser Meinung ist im Übrigen auch Mommsen (1856, S. 572 ff.), der von der Korrespondenz sagt, dass sie am ehesten noch das Bild von Cicero widergebe, ansonsten aber einen ziemlich negativen Eindruck von Cicero zu haben scheint. Vor diesem Hintergrund und der Tatsache, dass die Briefe Ciceros ein so breites Themenfeld abdecken, ist es verwunderlich, dass auf dessen deutschen Wikipedia-Seite den Briefen nur ein ganz kleiner Abschnitt gewidmet ist.

Was für ein Schatz italienische Dichter und Geschichtsschreiber Francesco Petrarca (1304–1374 n. Chr.) gehoben hatte, als er 1345 in Verona eine Abschrift von Ciceros Privatbriefen entdeckte, war ihm wohl kaum bewusst. Er schrieb die Briefe ab (und verfasste selbst zwei Briefe an Cicero (Egelhaaf-Gaiser, 2025, S. 9)) und sorgte so dafür, dass sie wieder in den Umlauf kamen. Dabei war Petrarca vom “neuen” Cicero, den er durch den Briefwechsel nun kennenlernen konnte, gelinde gesagt, enttäuscht. Verkörperte Cicero vor der Lektüre der Briefe für Petrarca durchaus ein Idol, so stellte sich Cicero für ihn in der Korrespondenz als überaus wankelmütig und, in Petrarcas Worten, unmännlich dar (Viertel, 1879, S. 8).

Nichtsdestotrotz dienten Ciceros Briefe vortan auch als Quelle der Inspiration. So beeinflussten sie die Humanismus-Bewegung während der ab dem 14. Jhdt. einsetzenden Renaissance massgebend (Walter, 2014, S. 43). Als solches

sicherte Petrarcas Einsatz das Überleben der römischen Brieftradition. Dies hatte zur Folge, dass die Cicero-Briefe seit deren Wiederentdeckung immer wieder Gegenstand der Forschung innerhalb verschiedener Disziplinen wie zum Beispiel in der altphilologischen Briefforschung, der klassischen Geschichtsforschung oder auch der Philosophie wurden. Einen ernsthaften Aufschwung erlebte die altphilologische Briefforschung jedoch erst in den 1980er Jahren, da Konzepte aus den Neuphilologien grossflächig auf die Altphilologie übertragen wurden (Egelhaaf-Gaiser, 2025, S. 5).

Die über 900 erhaltenen Cicero-Briefe sind nicht zuletzt darum interessant, weil eine Vielzahl davon wohl nicht für die Publikation vorgesehen war. Es ist selbstverständlich, dass man anders schreibt, wenn man weiss, dass nicht nur diejenige Person einen Brief lesen wird, an welche der Brief adressiert ist, sondern wohl noch einige 1'000 weitere. Davon zeugt, dass in vielen Briefen Grussformeln, Aufenthaltsorte und Informationen über die Gesundheit einer Reihe von Personen enthalten sind, was für zu publizierendes Material unüblich war (Manuwald, 2025, S. 799). Darüber hinaus wurden die Briefe nicht mehr zu Lebzeiten Ciceros publiziert, sondern erst posthum, wohl zu Zeiten Augustus' und Neros (Manuwald, 2025, S. 796). Dass Teile des Briefwechsels über 100 Jahre nach Ciceros Tod veröffentlicht wurden, heisst auch, dass nicht der später freigelassene Sklave und Sekretär Ciceros, Marcus Tullius Tiro (103–4 v. Chr.), die Briefe in Umlauf brachte, sondern ein nicht bekannter Editor, und das anhand weitgehendst unbekannter Kriterien (White, 2010, S. 61).

Für die Computerlinguistik sind Ciceros Briefe ein Glücksfall. Sie stellen eine Art riesige Spielwiese dar, auf welcher wir uns mit verschiedensten Methodiken "austoben" können. Dass bereits sehr viel Forschung zum Cicero-Briefwechsel vorliegt, ist dabei kein Hindernis. Vielmehr bieten die bereits bestehenden Erkenntnisse zur Korrespondenz Inspiration und valide Ausgangspunkte. So können bereits aufgestellte Theorien mit automatischen Verfahren überprüft und verifiziert oder verworfen werden. Aber auch neue Erkenntnisse lassen sich durch computerlinguistische Methoden gewinnen. Ein Forschungsdesiderat, das wiederholt genannt wird, ist die Netzwerkanalyse (Egelhaaf-Gaiser, 2025, S. 26). Damit werden wir uns im Kapitel ?? beschäftigen. Mit wem interagierten Cicero und sein persönliches Umfeld? Varianten des sogenannten *Distant Readings* (Moretti, 2013) (siehe Kapitel ??) helfen uns dabei, einen Überblick zu gewinnen. So spüren wir das Netzwerke Ciceros auf. Dazu verwenden wir auch die sogenannte *Eigennamenerkennung* (Eng.: *Named Entity Recognition*). Diese ist ein automatisches Verfahren, welche automatisch Personen, Orte, Institutionen und Toponyme im Briefwechsel markiert. Wenden wir darüber hinaus die *Sentimentanalyse* an, also das Identifizieren von Emotionen, können wir erschliessen, in welchen Beziehungen die Personen zueinander standen. Eine zusätzliche Auswertung des verwendeten Vokabulars, konkreter aber noch der Sprache an sich, wird uns zeigen, wie sich Cicero und seine Korrespondentinnen ausdrückten. Die Sprache wird im Allgemeinen von Interesse sein und so erhalten auch altphilologische Fragestellungen Einzug in unseren Themenstrass. Cicero hat beispielsweise immer wieder griechische Wörter in seinen Briefen eingebaut, also sogenanntes Code-Switching betrieben. Wann macht er das in der Regel? Wie häufig ist das

generell und mit welchen Motiven tut er das? Eine automatische Auswertung kann uns dabei helfen. Und zuguterletzt (obwohl das Ende der Fahnenstange noch lange nicht erreicht ist): Lassen sich die Briefe automatisch übersetzen, da die Lateinkenntnisse vieler Historiker:innen immer weiter abnehmen? Wenn ja, wie gut? Mit solchen Herangehensweisen hoffen wir, mit diesem Reader die Geschichtswissenschaften und die Computerlinguistik in Einklang zu bringen und damit die Geschichtsforschung als solche weiterzubringen.

1.3 Aufbau des Readers

Zu Beginn jedes Kapitels werden jeweils die Lernziele erklärt. Als Nächstes folgt der Inhalt, manchmal ergänzt durch Exkurse oder Kompakt-Abschnitte.

Lernziele Session 1: Einführung

Die Lernziele sind in blauen Boxen.

Exkurs

Die Exkurse sind in grünen Boxen.

Kompakt

Die Kompakt-Abschnitte sind in roten Boxen.

Wir orientieren uns im Kurs am Buch *Cicero in Letters* von White (2010). White nähert sich Cicero, ausgehend vom Briefwechsel selbst, in unterschiedlichsten Facetten. So beleuchtet er im ersten Teil seines Buches den Kontext, in welchem Ciceros Briefe entstanden sind, und erläutert die Rolle der schriftlichen Kommunikation sowie deren Hürden im Allgemeinen. Hier geht es darum, welche Rolle die Boten spielten, die die Briefe überbracht haben, und wie und wer den Briefwechsel so ediert hat, wie er uns heute vorliegt. Im zweiten Teil geht White zur Analyse des Inhaltes über. Hier kommen diskurslinguistische Elemente zum Tragen: Wie kommuniziert Cicero mit wem? Was spielt die literarische Kultur in der Kommunikation eine Rolle? Weiter geht White auch Ciceros ratgebende Korrespondenz ein. Hier interessieren ihn vor allem soziolinguistische Aspekte. Zuguterletzt widmet sich White auch dem Thema, wie Ciceros Rolle als politische Führungspersönlichkeit im Briefwechsel durchdringt. Somit liefert Whites Beitrag einen spannenden Einblick in Ciceros Briefpersona, die wir so anhand seiner Vorleistungen auch digital erschliessen können. Zeigen sich beispielsweise in seinen Briefen, in denen er Ratschläge gibt, gewisse Emotionen, oder erteilt er diese nüchtern und sachlich? Wo wird Cicero überhaupt emotional? Die sogenannte automatische *Sentimentanalyse* kann uns dabei unterstützen.

Als ergänzende Lektüre empfehlen wir *Cicero* (Manuwald, 2014) und das *Handbuch Brief: Antike* (Becker et al., 2025). Die Themen, die Peter White in seinem Buch zum Briefwechsel Ciceros behandelt, bilden die Grundlage der Struktur des Readers. So sind den Kapiteln Whites der Reihe nach die wichtigsten Themen entnommen, die man digital aufbereiten und ergründen kann. Somit

bietet es sich an, White und diesen Reader parallel zu lesen.

Aufgrund der aus dem Buch von White erschlossenen Inhalte sind die ab Kapitel 2 aufgeführten Beispiele entstanden. Diese sind so gegliedert wie in Code-Beispiel 1.1 dargelegt. Sie sollten die Funktionsweise des Besprochenen illustrieren. Am besten schreibt man die Code-Beispiele kurz ab, damit man die Konzepte bereits zum ersten Mal festigt. Zur Not können die Beispiele aber auch so in Jupyter Notebooks kopiert und ausgeführt werden.

Code-Beispiel 1.1 Beispiel eines Code-Abschnittes

```
print('Marcus Tullius Cicero te salutat!')  
Marcus Tullius Cicero te salutat!
```

Die vollständigen Jupyter Notebooks sind auf unserer [JupyterLite-Instanz](#) abgelegt. Dort finden sich auch sämtliche Daten. Der Code ist in dieser Browser-Umgebung vollständig ausführbar.

Nebst den Daten und Jupyter Notebooks möchten wir auch auf das [Wiki](#) aufmerksam machen, das im Zuge der Durchführung des Kolloquiums [Cicero Digital](#) am [Historischen Seminar](#) der [Universität](#) Zürich entstanden ist. Dort gestalteten die Studierenden in Partnerarbeit eine Wiki-Seite. Ziel war es, mit dem Cicero-Briefwechsel als Grundlage eine historische Fragestellung auszuarbeiten, welche mit komputationellen Methoden erschlossen werden konnte. Das Resultat ist eine Vielfalt von neuen Perspektiven, die sich aufgrund des andersartigen Zugangs auftun.

Der Brief als Datenbasis

Lernziele Session 2: Der Brief als Datenbasis

Verstehen Du verstehst, warum Ciceros Briefe als *Datenbasis* und nicht nur als historische “Quelle” behandelt werden können, und was White mit den *constraints and biases* der Überlieferung und Sammlung meint.

Analysieren Du kannst in einem Briefkorpus (Perseus) die für eine strukturierte Analyse relevanten Informationstypen identifizieren (z.B. Absender, Empfänger, Datum, Ort) und typische Probleme der Briefe als Daten benennen (Unvollständigkeit, Uneindeutigkeit, Editionsentscheidungen).

Anwenden Du kannst Briefe aus dem Perseus-Korpus in eine tabellarische Struktur überführen und ein einfaches Metadaten-Schema praktisch ausfüllen (mindestens Absender, Empfänger, Datum, Ort) sowie getroffene Normalisierungen und Entscheidungen dokumentieren.

Bewerten Du kannst die Aussagekraft deiner Metadaten kritisch einschätzen: Welche Felder sind stabil, welche unsicher? Wie beeinflussen *constraints and biases* (White) spätere Visualisierungen und Auswertungen?

Erzeugen Du kannst ein kleines, reproduzierbares Mini-Dataset (Tabelle + kurze Dokumentation) erstellen, das als Grundlage für weitere digitale Analysen dient (z.B. erste Diagramme oder spätere Netzwerk- und Raumanalysen).

2.1 Briefe in der Antike

Die Epistolografie – vom Griechischen *ἐπιστολή* (Brief) und *γράφειν* (schreiben, zeichnen), was aber als -kunde aufgefasst werden muss – ist also die Briefkunde, die Wissenschaft vom Briefeschreiben. Dabei geht sie, wie wir in Unterkapitel 1.2 gesehen haben, bereits bis in die Antike zurück.

Umso erstaunlicher ist es, dass gemäss Egelhaaf-Gaiser (2025, S. 5) die Auseinandersetzung mit der ‘Literaturgattung’ Brief in der altphilologischen Forschung erst seit den 1980er-Jahren an Attraktivität gewonnen hat. Dennoch bleibt es vorerst zu definieren, was man unter ‘Brief’ im eigentlichen Sinne versteht. Man würde doch meinen, dass diese Frage im 21. Jahrhundert geklärt ist. Sieht man aber genauer hin, eröffnen sich auch hier Schwierigkeiten der Begriffsdefinition ‘Brief’.

Die ersten Assoziationen, die einem vor das innere Auge treten, wenn man an einen Brief denkt, sind wohl ein (von Hand?) geschriebenes Dokument, das in einem Briefumschlag von einer Person A–dem Sender–einer anderen Person B–dem Empfänger–übermittelt wurde. Würden wir das als Briefdefinition annehmen, böte eine solche Festlegung für die Moderne bereits genügend Angriffsfläche: Ist eine Nachricht auf WhatsApp ein Brief? Was ist mit dem Mailverkehr? Und wie sieht das mit Beiträgen auf Social-Media-Kanälen aus? Wie öffentlich können Briefe sein? Welche Elemente müssen sie enthalten, um als Brief zu gelten? Und was ist nur optional?

Zumindest mit Blick auf die Antike gestaltet sich die Frage nach dem Medium etwas einfacher, obwohl es auch damals bereits eine gewisse Vielfalt gab. So konnten Briefe auf *Ostraka* (also Tonscherben) oder auf Papyrus, Holz- oder Tontafeln übermittelt werden. Diese Liste ist sicher nicht abschliessend und um die eigentliche Materialität soll hier nicht zum Thema werden. Ähnlich schwierig wie für ‘moderne’ Briefe gestaltet sich jedoch die Frage, wie antike Briefe inhaltlich definiert und klassifiziert werden können.

Neben (vermutlicherweise) Demetrios finden wir auch bei anderen antiken Autoren Hinweise für eine Gattungsdefinition. So schreibt zum Beispiel der Kirchenvater Augustinus (354–430 n. Chr.) in seinen *Überarbeitungen*, dass sein Buch *Ad inquisitiones Ianuarii* (“Auf die Anfragen des Ianuarius”) eigentlich ein Brief ist, da zu Beginn steht, wer an wen schreibt (*retr.* 2,20). So finden wir “Auf die Anfragen des Ianuarius” heute nicht unter seinen Werken, sondern in seinem Briefwechsel. Augustinus bediente sich insofern bestimmter Kriterien (wie etwa dem Vorhandensein von Absender und Empfänger), um seine Schriften zu klassifizieren.

Briefe wurden von antiken Autoren aber auch noch genauer umschrieben. So treffen wir immer wieder auf den Ausdruck des “halben geschriebenen Dialog” (in der Antike in Demetr. *eloc.* 223 f., aufgegriffen ebenfalls bei Seibert, 2025, S. 226 und Ebbeler, 2010, S. 468) zwischen zwei Personen *in absentia* (Cic. *Phil.* 2,7). Hierbei ist das Konzept der Hälfte entscheidend: denn es ist gerade bei den Cicero-Briefen so, dass die eine Hälfte, und damit ist hier die grösstenteils an Cicero gerichtete Korrespondenz gemeint, fehlt. Das ist für die Antike typisch (Hoskin, 2025, S. 88). So finden wir im Cicero-Briefwechsel nur etwa gerade 10% an Briefen, in denen Cicero als Empfänger fungierte (White, 2010, S. 6). Diese Situation wurde nur dann bereits in der Antike entschärft, wenn die Briefe von Beginn an zur Publikation gedacht waren, wie das bei **Plinius dem Jüngeren** (61/62–113/115 n. Chr.).

Die Briefe Ciceros sind aber auch ganz generell eine Quelle für die Brieftheorie (Becker & Blom, 2025, S. 134). So widmet sich beispielsweise Schwitter

(2017) den Themen Beileid und Trost–auf das wir im Übrigen in Kapitel ?? nochmals zurückkommen werden–in den *Epistulae ad familiares*, insbesondere bei Todesfällen (*de morte*). Schwitter arbeitet heraus, wie Briefschreiber in der Antike ihre Anteilnahme verschriftlichten und welche Auswirkungen das auf die Empfänger gehabt haben muss. Die Briefe, in welchen vom Tod von Ciceros Tochter Tullia 45 v. Chr. die Rede ist, geben deshalb besondere Einblicke in die Biographie von Cicero, wie das im Übrigen auch für viele andere Briefe der Fall ist (Hoskin, 2025, S. 93).

Bei der Interpretation ist jedoch Vorsicht geboten. Bereits Schwitter merkt an, dass “die Sammlung der *Epistulae ad familiares* lediglich einen unvollständigen Einblick in die Praxis brieflicher Trost- und Beileidsschreiben *de morte* bietet” (2017, S. 377), also kein abschliessendes Urteil über die Trauerbewältigung und den Trost als solches möglich ist. Schliesslich kamen die Briefe auch in einer mehr oder weniger aufwändig editierten Version auf uns, wobei wir über einiges im Dunkeln sind: Wir wissen nicht, *wer* das Cicero-Briefkorpus kompiliert hat. Wir wissen nicht, *wie* die unbekannten Editoren in den Publikationsprozess eingegriffen haben. Wie viel (wenn überhaupt) wurde am Text geändert? Wir wissen nicht, wie oft diese Publikationen weiter abgeschrieben und verändert worden sind.

Und trotzdem erfahren wir aus Ciceros Korrespondenz so viel über den antiken Brief wie selten irgendwo, da er oft davon spricht “wann und nach welcher Zeitspanne ein Brief ihn erreicht habe, auf welchem Wege er Briefe abschicke oder erhalte, ob die Zustellung gewährleistet sei, da Briefe unterwegs verloren gehen, geöffnet oder abgefangen werden oder Kuriere nicht zuverlässig sein könnten, oder dass er zur Sicherheit mit eigener Hand oder auf Griechisch schreibe” (Manuwald, 2025, S. 799). In der Interaktion zweier Abwesener, wie Cicero gemäss Rühl (2025, S. 328) zu sagen pflegte, kristallisieren sich die Beziehungen zwischen Absendern und Empfängern heraus, welche der Absender weiter zu formen versuchte, nämlich durch das Aufzeigen von “Status, Bildung, Verbindungen und Einflussmöglichkeiten” (Seibert, 2025, S. 230).

Es ist deshalb unabdingbarerweise der Inhalt des Briefes an die Form auf der einen Seite und die Funktion auf der anderen Seite geknüpft. Das heisst konkret, dass die von den Verfassern verwendete Sprache ein Ziel verfolgte, beispielsweise das von Schwitter (2017) untersuchte Trösten des physisch nicht präsenten Gegenübers. Die literarische Form des Briefes kann jedoch unterschiedlich ausgeprägt sein. Das Spektrum reicht von strikt historisch bis zu poetisch, wobei die Grenzen fließend sind (Ebbeler, 2010). Um Form und Funktion auf den Grund gehen und jedem Brief seine Wirkung zuordnen zu können (Egelhaaf-Gaiser, 2025, S. 10), müssen wir zuerst an die Inhalte gelangen. Das Briefkorpus Ciceros ist längst auf verschiedenen Webseiten im Netz zu finden. In den nächsten zwei Unterkapiteln gehen wir auf die ersten Vorverarbeitungsschritte ein.

2.2 Die Datengrundlage bei Cicero

Die Datengrundlage, das heisst, die Menge an Briefen von Cicero, die uns überliefert sind, ist für antike Verhältnisse eigentlich als “gut” einzustufen. Die

Rede ist oft davon, dass die Spätantike ergiebiger sei. Das mag für die Anzahl der Korrespondenzen zutreffend sein—man denke an Augustinus, **Gregor von Nazianz** (329–390 n. Chr., über 200 überlieferte Briefe), oder **Isidorus von Pelusium** (360–431/451 n. Chr., von ihm kennen wir über 2'000 Briefe), um nur einige zu nennen—, aber vergleicht man beispielsweise diejenige des Augustinus mit der von Cicero, kann Augustinus lediglich mit etwas mehr als 300 Briefen aufwarten.

Ganz abgesehen von der Menge an Briefen, benötigen wir für die automatische Verarbeitung eine digitale Fassung der Briefe. Wir gehen deshalb kurz auf unseren “Datenlieferanten” Perseus ein (siehe Unterkapitel 2.2.1). Die Daten liegen uns in sogenannten XML-Dateien vor. Das Format beschreiben wir im Unterkapitel 2.2.2 genauer. Abschliessend widmen wir uns im Abschnitt ?? den ersten Verarbeitungsschritten. Hierbei führen wir anhand von Jupyter-Notebooks durch die ersten einfachen Analysemethoden.

2.2.1 Perseus

Die Texte von Cicero stammen aus dem dem *Perseus Digital Library Project*, das seit 1985 darum bemüht ist, Texte aus der griechischen und römischen Antike online zur Verfügung zu stellen (Crane, 1998). Dabei geht das **Online-Portal** über die bloße Bereitstellung der rohen Texte hinaus. Über die Jahre hinweg wurden die griechischen und lateinischen Texte nicht nur digital aufbereitet (Smith et al., 2000), sondern auch mit automatischer Wortartenannotation und Parsing-Informationen versehen, sodass veritable Baumbanken entstanden sind (Bamman et al., 2007). Auch im Bereich der sogenannten *Linked (Open) Data* ist Perseus gut vertreten (Babeu & Dilley, 2021).¹

Für die vier Subkorpora *ad familiares*, *ad Atticum*, *ad Quintum fratrem* und *ad M. Brutum* findet man im **GitHub-Repository** die relevanten XML-Dateien: `phi0474.phi056.perseus-lat1.xml` (*ad familiares*), `phi0474.phi057.perseus-lat1.xml` (*ad Atticum*), `phi0474.phi058.perseus-lat1.xml` (*ad Quintum fratrem*) und `phi0474.phi059.perseus-lat1.xml` (*ad M. Brutum*).

Um die Briefe im Übrigen auffinden zu können, wie sie in der Literatur angegeben sind, empfiehlt es sich, den **Scaife Viewer** zu verwenden. Wenn White also beispielsweise *Fam.* 6.15 zitiert, meint er Buch 6 in *ad familiares*, Brief 15. Leider finden wir aber nur das erste Buch aus *ad familiares* im Scaife Viewer, weshalb wir auf Perseus durchzählen müssen.

2.2.2 Datenformat: XML

Viele Daten, mit denen auch in der Geschichtsforschung gearbeitet wird, liegen uns im sogenannten XML-Format vor. XML steht für *eXtensible Markup Language*. Sie ist eine Auszeichnungssprache, die wir in Textdateien verwenden und üblicherweise mit der `.xml`-Endung abspeichern. XML ermöglicht es uns, Daten hierarchisch zu strukturieren.

¹Für eine umfassende Publikationsliste, siehe <https://www.perseus.tufts.edu/hopper/about/publications>.

Der Standard hat sich seit 1998, als er zum ersten Mal veröffentlicht wurde, etabliert und stetig weiterentwickelt.² Für Textdaten hat sich zudem das sogenannte TEI-XML durchgesetzt. TEI steht für *Text Encoding Initiative*.³ Darauf bauen weitere Standards für verschiedenste Textgattungen auf. EpiDoc⁴ ist beispielsweise eine an TEI angelehnte Version, welche das Erfassen von Inschriften vereinfacht. Liegen die Daten einmal in einem standardisierten XML-Format vor, so ist es ein Leichtes, sie miteinander zu teilen, die Annotationen zu erweitern oder zu veröffentlichen.⁵ Zudem können sie mit bereits existierenden und extra dafür konzipierten eingelesen und maschinell verarbeitet werden.

Als Motivation, auf etablierte Annotationsschemata wie (TEI-)XML zurückzugreifen, steht der *FAIR*-Grundsatz. FAIR ist kurz für:

1. **Findable** wenn wir Daten veröffentlichen, wollen wir das so tun, dass man sie einfach findet.
2. **Accessible** wir wollen, dass die Daten maschinenlesbar sind.
3. **Interoperable** Daten sollen in gängigen, offenen Formaten vorliegen und mit anderen Systemen kompatibel sein.
4. **Reusable** die Daten sollen einfach wiederzuverwenden sein.

Ein weiterer Vorteil, den XML-Dateien bieten, ist deren hoher Grad an Standardisierung. Standards liegen sogenannte *Schemata* zugrunde. Sie definieren, wie eine XML-Datei strukturiert sein muss. Das Praktische daran: Man kann im Zuge der Erstellung einer XML-Datei wiederholt gegen das Schema prüfen und so in Erfahrung bringen, ob die erzeugte Datei *valide* (also gültig) ist. Im gegenteiligen Fall verläuft die Validierung mit einem Fehler, den man anschliessend korrigieren sollte. So stellt man sicher, dass man stets dem Schema folgt und gültiges XML produziert. Das ist wichtig, da ungültige XML-Dateien nicht eingelesen werden können, auch nicht von Python.

Wenden wir uns nun aber dem Aufbau einer XML-Datei zu. Für gewöhnlich steht an erster Stelle einer XML-Datei die Deklaration. Hier spezifizieren wir, um was für eine XML-Datei es sich handelt und welche Version wir verwenden. Es folgen einige Beispiele von Deklarationen:

Code-Beispiel 2.1 Beispiel 1 für eine XML-Deklartion.

```
<?xml version="1.0" encoding="UTF-8"?>
```

Code-Beispiel 2.2 Beispiel 2 für eine TEI-XML-Deklartion.

```
<TEI xml:lang="en" xmlns="http://www.tei-c.org/ns/1.0">
```

²Für weitere Informationen, siehe die Homepage des World Wide Web Consortiums (W3C): <https://www.w3.org/XML/>.

³Eine umfassende Dokumentation dieses Standards ist auf <https://tei-c.org/> zu finden.

⁴Siehe <https://epidoc.stoa.org/>.

⁵Zum Beispiel mit dem *TEI-Publisher*, siehe <https://teipublisher.com/exist/apps/tei-publisher-home/index.html>.

General period	Metal	Weight	Denomination	Date from	BCorAD1	Date to	BCorAD2	Reverse type	Catalogue reference
4th Century AD Roman	Copper Alloy		Nummus	337	AD	340	AD	Uncertain	Uncertain
3rd Century AD Roman	Copper Alloy		Radiate	260	AD	269	AD	Uncertain	Uncertain
3rd Century AD Roman	Billon		Radiate	251	AD	253	AD	Uncertain	Uncertain
Augustus & 1st Century AD Roman	Copper Alloy		Dupondius	43	AD	64	AD	CERES AVGVSTA SC	as RIC I Claudius 110
Augustus & 1st Century AD Roman	Copper Alloy		Dupondius	43	AD	64	AD	CERES AVGVSTA SC	as RIC I Claudius 110
Augustus & 1st Century AD Roman	Copper Alloy		As	43	AD	64	AD	Minerva SC	as RIC I Claudius 100

Tabelle 2.1 Die ersten sechs Zeilen und einige ausgewählte Spalten des Münzdatensets aus Wales.

In den recht kurzen Code-Beispielen 2.1 und 2.2 passiert schon recht viel. Im Ersten definieren wir, dass der folgende Inhalt eine XML-Datei darstellt. Zusätzlich geben wir die Version mit dem Attribut-Namen `version` und die Zeichenkodierung mit `encoding` an. Im Letzteren spezifizieren wir den Inhalt als TEI. Gleichzeitig sagen wir mit `xml:lang`, dass der Text der Datei auf Englisch gehalten ist. `xmlns` definiert den *Namespace* und weist darauf hin, dass die Elemente in dieser XML-Datei zum TEI-Standard gehören.

Was sind nun die Zutaten, die wir für die Repräsentation von Daten in XML brauchen?

1. **Elemente** Elemente sind die Baublöcke von XML-Dateien. Sie erhalten einen Namen (der theoretisch frei wählbar ist) und werden zwischen spitze Klammern `<...>` gefasst. Elemente können in sich geschlossen sein, z. B., `<coin/>` (beachte den Forwardslash vor der abschliessenden Klammer), oder als Start- und End-Tag aufgebaut sein: `<coin>Dupondius</coin>`. Das End-Tag wird hier mit dem Forwardslash markiert und muss gleich heissen wie das Start-Tag.
2. **Attribute** Elemente können Attribute enthalten. Diese spezifizieren das Element genauer. Attribute sind wie Key-Value-Paare in Python-Dictionaries aufgebaut. Sie müssen mit einem Buchstaben oder einem Unterstrich starten. Der Wert muss zwischen doppelten Anführungszeichen stehen und durch ein Gleichheitszeichen abgetrennt sein. Als Beispiel: `<coin denomination="Dupondius"/>`.
3. **Hierarchie** Wie bereits erwähnt, lässt sich die Datenstruktur mit XML hierarchisch repräsentieren. Elemente können beliebig tief verschachtelt werden. Allerdings muss die Reihenfolge beachtet werden. Wenn die Elemente 1, 2 und 3 geöffnet werden, müssen sie in umgekehrter Reihenfolge geschlossen werden, also zuerst 3, dann 2 und schliesslich 1. Würde man nach 3 zuerst 1 schliessen wollen, wäre das XML ungültig.

Wir bedienen uns für die Illustration einer ersten, ganz einfachen XML-Struktur eines Datensatzes aus der Antike: *Iron Age and Roman Coins from Wales* von der University of Wales. Das Datenset ist frei auf *Archaeology Data Service* verfügbar.⁶ Der Datensatz (als .csv-Datei) sieht unter der Weglassung einiger Spalten aus wie in Tabelle 2.1. Eine Münze aus diesem Datenset finden wir in Abbildung 2.1.

Wir verwenden jetzt die oben erwähnten Zutaten und adaptieren also erstes einmal eine Münze als XML-Struktur:

⁶Siehe <https://doi.org/10.5284/1000263>.



(a) Vorderseite



(b) Rückseite

Abbildung 2.1 Münze des Nero (As) aus dem Jahr 65 n. Chr. (RIC I Nero 416).**Code-Beispiel 2.3** Teil eines Münzdatensets als XML-Struktur.

```
<?xml version="1.0" encoding="UTF-8"?>
<coins>
  <coin id="6">
    <weight unit="g">5.41</weight>
    <denomination>Dupondius</denomination>
    <issuer>Claudius I</issuer>
    <date from="43" to="64"/>
    <reverse_type>CERES AVGVSTA SC</reverse_type>
  </coin>
  <coin id="4">
    <weight unit="g">6.49</weight>
    <denomination>As</denomination>
    <issuer>Agrippa (Gaius)</issuer>
    <date from="43" to="64"/>
    <reverse_type>Neptun SC</reverse_type>
  </coin>
</coins>
```

Wir sehen, wie die verschachtelte Struktur funktioniert. Wir können uns das Element `<coins>` als Liste vorstellen, in welcher wir alle Münzen sammeln. Jeder `<coin>` in dieser Liste ist eindeutig durch seine `id` identifizierbar. Was darauf folgt, sind weitere Eigenschaften der jeweiligen Münze. Der Datensatz wäre auch viel flacher strukturierbar:

Code-Beispiel 2.4 Teil eines Münzdatensatz als XML-Struktur.

```
<?xml version="1.0" encoding="UTF-8"?>
<coins>
  <coin id="6" weight="5.41g" denomination="Dupondius" issuer="
Claudius I" date_from="43" date_to="64" reverse_type="CERES AVGVSTA
SC"/>
  <coin id="4" weight="6.49g" denomination="As" issuer="Agrippa (
Gaius)" date_from="43" date_to="64" reverse_type="Neptun SC"/>
</coin>
</coins>
```

Code-Beispiel 2.4 zeigt, wie im Prinzip alle Elemente aus Code-Beispiel 2.3 auch als Attribute definiert werden können. Das erfordert jedoch eine leichte Umstrukturierung (beispielsweise des Datums oder des Gewichts). Benutzen wir den XML-Standard, sind wir bei der Gestaltung der Datenstruktur relativ frei.

In der Numismatik hat sich jedoch das *Numismatic Description Schema* (NUDS) etabliert. Es orientiert sich ebenfalls am TEI-Standard. Ein einfacher Eintrag für bloss eine Münze kann mit diesem Schema auf gut über 100 Zeilen anschwellen. Vor allem den Metadaten kommt ein grösseres Gewicht zu. Hier ist ein Auszug einer Münze aus dem 2. Jhr. v. Chr., die den Herakles zeigt:⁷

Code-Beispiel 2.5 Münze ‘Price 2051’ im NUDS-XML-Format. Die Punkte zu Beginn und am Ende dieses Beispiels weisen auf die Unvollständigkeit hin.

```
...
<typeDesc>
  <objectType xlink:type="simple" xlink:href="http://nomisma.
  org/id/coin">Coin</objectType>
  <dateRange>
    <fromDate standardDate="-0200">200 BCE</fromDate>
    <toDate standardDate="-0196">196 BCE</toDate>
  </dateRange>
  <denomination xlink:type="simple" xlink:href="http://
  nomisma.org/id/tetradrachm">Tetradrachm</denomination>
  <manufacture xlink:type="simple" xlink:href="http://nomisma
  .org/id/struck">Struck</manufacture>
  <material xlink:type="simple" xlink:href="http://nomisma.
  org/id/ar">Silver</material>
  <authority>
    <persname xlink:type="simple" xlink:role="
  statedAuthority" xlink:href="http://nomisma.org/id/alexander_iii">
  Alexander III of Macedon</persname>
  </authority>
  <geographic>
    <geogname xlink:type="simple" xlink:role="mint"
  xlink:href="http://nomisma.org/id/magnesia_ad_maeandrum">Magnesia ad
  Maeandrum</geogname>
  </geographic>
  <obverse>
    <type>
      <description xml:lang="en">Head of beardless
  Heracles right wearing lion skin headdress</description>
      <description xml:lang="de">Kopf des jugendlichen
  Herakles nach r. mit Löwenfellexuvie</description>
      <description xml:lang="fr">Tête d'Héraclès imberbe
  à dr. coiffé d'une dépouille de lion</description>
    </type>
    <persname xlink:type="simple" xlink:role="deity"
  xlink:href="http://nomisma.org/id/heracles">Heracles</persname>
  </obverse>
  ...
```

Das Code-Beispiel 2.5 zeigt die Beschreibung der Münze, nachdem die Metadaten definiert wurden. Wir sehen eine *dateRange* mit den Werten “200

⁷Siehe <https://www.numismatics.org/pella/id/price.2051>.

BCE” bis “196 BCE” oder auch die uns bereits bekannte denomination (hier “Tetradrachme”). Es sind überdies noch material, authority sowie geografische Angaben zum Prägungsort und zur Bildseite der Münze gegeben. Zudem sehen wir viele Attribute, meist mit dem Präfix xlink:. Der Wert dieses Attributs ist dann ein Link, der auf eine Konzept-Seite in NUDS verweist. Beispielsweise verlinkt http://nomisma.org/id/alexander_iii auf die Konzept-Seite Alexander des Grossen (siehe Abbildung 2.2).

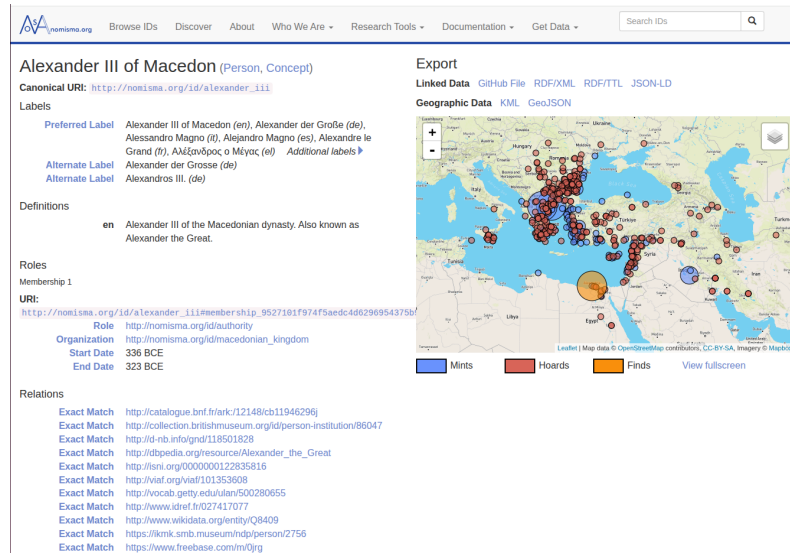


Abbildung 2.2 Screenshot der Personenseite Alexander des Grossen auf Numisma.

Da alle Münzen, die unter Alexander dem Grossen geprägt wurden, mit xlink: auf diese Seite verweisen, findet man weiter unten eine Sammlung aller Münzen unter Alexander dem Grossen. Das ist ein Beispiel von *Linked Open Data*, die sich dadurch auszeichnet, dass ein Datensatz mit weiteren Datensätzen aus dem Netz verbunden ist (siehe dazu die “Relations”, die auf Konzeptseite Alexander des Grossen zu finden sind). Dies zeigt die Nützlichkeit der Strukturierung der Daten in XML.

Als Nächstes schauen wir uns noch das eingangs erwähnte EpiDoc XML zur Auszeichnung von Inschriften an. Dafür nehmen wir uns ein Dokument aus den *Inscriptions of Roman Tripolitania 2021*⁸ zur Hilfe. Die Inschrift unter <https://irt2021.inslib.kcl.ac.uk/en/inscriptions/IRT0002.html> ist in TEI-XML strukturiert. TEI zeichnet sich in der Regel dadurch aus, dass zuerst ein `teiHeader`-Element folgt, das sämtliche Metadaten enthält. Im darauffolgenden `text`-Element ist dann der Text verzeichnet. Wenn wir uns einen Ausschnitt aus diesem XML anschauen (wir fokussieren uns auf die ersten fünf Zeilen), sieht dies eher kompliziert aus:

Code-Beispiel 2.6 Teil eines TEI-XMLs zu einer Inschrift.

```
<div type="edition" xml:lang="la" xml:space="preserve">
  <ab>
    <lb n="1"/>
    <persName type="divine" ref="#caelestis"><addName nymRef
      ="domina"><supplied reason="lost">Do</supplied>minae</addName>
```

⁸Siehe <https://irt2021.inslib.kcl.ac.uk/en/>.


```

<name nymRef="Caelestis">Caelesti</name></persName>
<lb n="2"/><w lemma="pro"><supplied reason="lost">pro</
supplied></w> <w lemma="salus">salute</w>
<w lemma="imperator"><expan><abbr>Imp<del rend="erasure
">p</del></abbr><ex>eratorum</ex></expan></w>
<lb n="3"/><persName type="emperor" ref="#marcusaurelius
"><name type="praenomen" nymRef="Marcus"><supplied reason="lost"><
expan><abbr>M</abbr><ex>arci</ex></expan></supplied></name>
<name type="gentilicium" nymRef="Aurelius"><supplied
reason="lost">A</supplied>ureli</name>
<name type="cognomen" nymRef="Antoninus">Antonini</
name></persName> <del rend="erasure"><w lemma="et">et</w></del>
<lb n="4"/><persName type="emperor" ref="#commodus"><del
rend="erasure"><name type="praenomen" nymRef="Marcus"><expan><abbr>
M</abbr><ex>arci</ex></expan></name>
<name type="gentilicium" nymRef="Aurelius">Aureli</name
></del>
<name type="cognomen" nymRef="Commodus"><del rend="
erasure">Commod</del>i</name>
<lb n="5"/><addName nymRef="Augustus"><expan><abbr>Aug<
del rend="erasure">g</del></abbr><ex>ustorum</ex></expan></addName>
</persName>
<del rend="erasure"><w lemma="et"><supplied reason="lost
">e</supplied><unclear>t</unclear></w>
<persName type="emperor" ref="#crispina"><name type="
gentilicium" nymRef="Bruttia"><supplied reason="lost">Brutt</
supplied><unclear>i</unclear><supplied reason="lost">ae</supplied></
name>

```

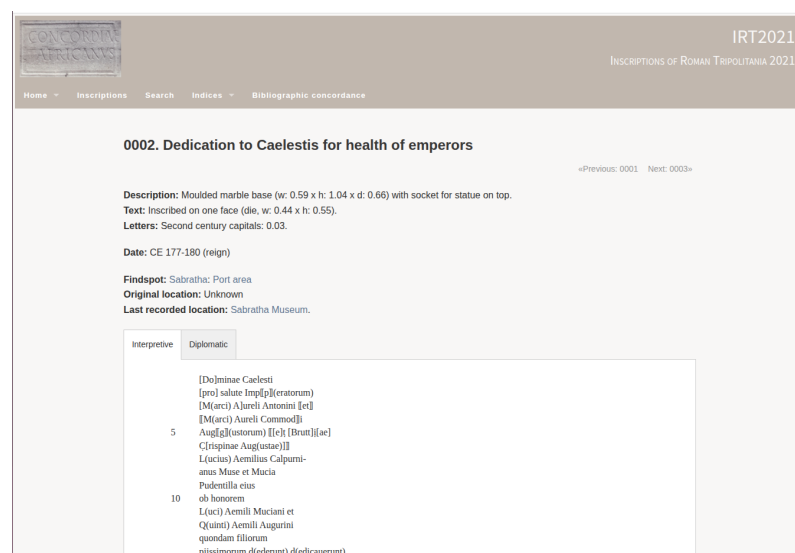


Abbildung 2.3 Repräsentation der Inschrift auf der Seite zu *Inscriptions of Roman Tripolitana*.

Wir sehen, dass die eigentliche Edition des Textes mit einem `div`-Element beginnt. Danach folgen die einzelnen Zeilen, separiert durch nummerierte Zeilenumbrüche (`lb`). Nun wird der Text wiedergegeben, wobei dieser kräftig

ausgezeichnet ist. Ein Blick auf die Repräsentation dieser Inschrift auf der offiziellen Homepage hilft (siehe Abbildung 2.3). Zudem ist ein Foto der tatsächlichen Inschrift für ein besseres Verständnis hilfreich (siehe Abbildung 2.4).

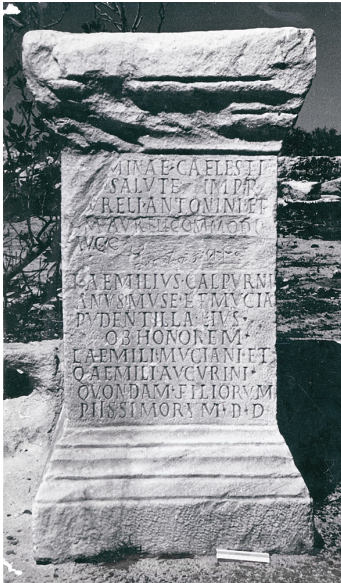


Abbildung 2.4 Fotografie der Inschrift CIL VIII, 22689 aus Sabratah, Provinz *Africa proconsularis*, datiert zwischen 178 und 195 n. Chr.

Die erste Zeile im XML lautet `<persName type="divine" ref="#caelestis"><addName nymRef="domina"><supplied reason="lost">Do</supplied>minae</addName><name nymRef="Caelestis">Caelesti</name></persName>`. Wir sehen hier zu Beginn eine Beschädigung an einer Inschrift. Das “Do” ist verloren. Im XML wird dies mit einem `supplied`-Element und der Angabe des Grundes hinzugefügt. Auf der Homepage wird dies dann automatisch als “[Do]” gerendert, wobei dies bei der diplomatischen Umschrift als “[..]” wiedergegeben wird.

In Zeile drei wird es noch interessanter: `<persName type="emperor" ref="#marcusaurelius"><name type="praenomen" nymRef="Marcus"><supplied reason="lost"><expan><abbr>M</abbr><ex>ar</ex></supplied></name><name type="gentilicium" nymRef="Aurelius"><supplied reason="lost">A</supplied>ureli</name><name type="cognomen" nymRef="Antoninus">Antonini</name></persName><del rend="erasure"><w lemma="et">et</w>`. Hier ist die Rede vom Kaiser Mark Aurel (121–180). Es wird also der Name “Marci Aureli Antonini” getaggt, obwohl das “Marci” ergänzt werden muss. Gleichzeitig erhält der `persName` ein `ref`-Attribut. Dieses referenziert oder zeigt auf `#marcusaurelius`. Dass dieser String mit einem Hashtag beginnt, bedeutet, dass es in einem anderen Dokument, in dem zum Beispiel Personennamen gesammelt werden, ein Element mit der ID `xml:id="marcusaurelius"` gibt. Unter diesem Element sind dann alle weiteren Informationen wie Lebensdaten, Ernennung zum Kaiser und Links zu externen Wissensressourcen verzeichnet. Das ermöglicht es dann, eine Personenseite wie die Konzeptseite Alexander des Grossen von der Numismatik-Datenbank zu bauen. Wenn mit `ref` und den entsprechenden IDs auf andere XML-Dokumente verwiesen wird, spricht man von einer sogenannten *Stand-off Annotation*. Somit müssen wir nicht an Ort und Stelle alle Infos zu Mark Aurel wiedergeben, sondern können sie gesondert in einem anderen Dokument aggregieren und, wenn immer nötig, wieder ändern. In der Folge können wir immer dann, wenn Mark Aurel erneut in einer Inschrift vorkommt, einfach auf den Eintrag im Personenverzeichnis verweisen. Wichtig ist, dass die IDs eindeutig sind.

Als Letztes betrachten wir noch ein Beispiel aus dem Briefkorpus des Cicero, um den es ja letztendlich geht. Für das XML in Code-Beispiel 2.7 wurde Text aus *ad M. Brutum* entnommen.

Code-Beispiel 2.7 Ausschnitt eines TEI-XMLs aus der Korrespondenz zwischen Cicero und Brutus.

```

...
<text xml:lang="lat">
  <body>
    <div xml:lang="lat" type="edition" n="
urn:cts:latinLit:phi0474.phi059.perseus-lat1">
      <div type="textpart" n="1" subtype="Book">
        <head>LIBER PRIMVS</head>
        <div type="textpart" n="1" subtype="letter">
          <label rend="opener">
            <seg rend="dateline">Scr. eodem die quo ep
. 2a. <date when="-0043">711 (43)</date>.</seg>
            <seg rend="salute">Cicero Bruto salutem</
seg>
              </label>
              <div type="textpart" n="1" subtype="section">
                <p>L. Clodius, tribunus plebis designatus, valde
me diligit vel, ut <foreign xml:lang="greek">e)mfatikw/teron</
foreign> dicam, valde me amat. <reg>quod</reg> cum mihi ita
persuasum sit, non dubito (bene enim me nosti) quin illum quoque
iudices a me amari. <reg>nihil</reg> enim mihi minus hominis videtur
quam non respondere in amore iis a quibus provocere. <reg>is</reg>
mihi visus est suspicari nec sine magno quidem dolore aliquid a suis
vel per suos potius iniquos ad te esse delatum quo tuus animus a se
esset alienior. <reg>non</reg> soleo, mi Brute, quod tibi notum
esse arbitror, temere adfirmare de altero; est enim periculosum
propter occultas hominum voluntates multiplicisque naturas; sed
Clodi animum perspectum habeo, cognitum, iudicatum. <reg>multa</reg>
eius indicia sed ad scribendum non necessaria. <reg>volo</reg> enim
testimonium hoc tibi videri potius quam epistulam. <reg>auctus</reg>
> Antoni beneficio est. <reg>eius</reg> ipsius benefici magna pars a
te est. <reg>itaque</reg> eum salvis nobis
          <pb/>
          vellet salvum.
        </p>
      </div>
    <div type="textpart" n="2" subtype="section">
...

```

Wir halten fest, dass die Briefe in div-Elementen mit texttype="letter" festgehalten werden. Hierbei finden wir den Text in div-Elementen, die als section ausgewiesen werden. Jeder Brief kann also mehrere Sektionen enthalten. Des weiteren finden, falls der Brief datiert ist, ein Segment seg mit einem rend-Attribut dateline. Zudem ist die Anrede als salute gekennzeichnet. Im Text finden wir nur spärliche Annotationen. Auffallend ist die Auszeichnung griechischer Passagen mit dem Element foreign. Zudem finden wir einige Wörter, die durch ein reg-Element ausgezeichnet wurden. Hierbei handelt es

sich um eine normalisierte Schreibweise einer Textstelle.⁹ Leider können wir nicht mehr nachvollziehen, was vor der jetzt im XML stehenden Form im Text stand.

2.3 Erste Verarbeitungsschritte

In diesem Abschnitt gehen wir vom TEI-XML der Perseus-Edition zu einem reproduzierbaren Dataset. Ziel ist es, die Briefe Ciceros nicht nur als “Texte”, sondern als strukturierte Datenbasis zu behandeln.

Am Ende dieses Prozesses steht eine Tabelle mit zentralen Metadaten pro Brief, die als Grundlage für weitere Analysen dient. Das dazugehörige Jupyter Notebook befindet in der [JupyterLite-Umgebung](#).

2.3.1 Dateien finden

Zunächst sammeln wir alle XML-Dateien aus dem Ordner `data/`. Die Annahme ist, dass sich dieser Ordner im selben Verzeichnis wie das Notebook befindet.

Code-Beispiel 2.8 Alle XML-Dateien im Ordner data sammeln.

```
from pathlib import Path

data_dir = Path("data")
xml_files = sorted(data_dir.glob("*.xml"))

len(xml_files)
```

Hier wird bereits ein zentraler Grundsatz deutlich: Statt einzelne Dateien manuell zu öffnen, automatisieren wir den Zugriff. Das ist eine Voraussetzung für Reproduzierbarkeit.

2.3.2 TEI und Namespace

Perseus verwendet TEI-XML. TEI-Dateien besitzen häufig einen sogenannten *Default-Namespace*. Wenn wir XPath-Abfragen durchführen, müssen wir diesen Namespace berücksichtigen.

Code-Beispiel 2.9 Namespace aus der XML-Datei ermitteln.

```
from lxml import etree

parser = etree.XMLParser(recover=True, huge_tree=True)
doc = etree.parse(str(xml_files[0]), parser)
root = doc.getroot()

root.nsmap
```

⁹Siehe dazu <https://tei-c.org/release/doc/tei-p5-doc/de/html/ref-reg.html> in der TEI-Dokumentation.

Wenn in `nsmmap` ein Eintrag für `None` erscheint, handelt es sich um den Default-Namespace. Diesen müssen wir später in XPath-Abfragen explizit angeben.

2.3.3 Was ist ein Brief im XML?

In den Perseus-Dateien sind einzelne Briefe als `div`-Elemente mit `subtype="letter"` ausgezeichnet.

Code-Beispiel 2.10 Alle Briefe in einer Datei finden.

```
letter_divs = root.xpath(
    ".//tei:div[@subtype='letter']",
    namespaces={'tei': root.nsmmap.get(None)}
)

len(letter_divs)
```

Wir arbeiten hier nicht mit dem sichtbaren Text, sondern mit der hierarchischen Struktur des XML-Dokuments.

2.3.4 Metadaten extrahieren

Ein Brief besitzt mehrere interessante Informationstypen:

- Buchnummer
- Briefnummer
- Datumszeile
- Datumsattribut `date/@when`
- Anrede

Beispielsweise lässt sich die Datumszeile so extrahieren:

Code-Beispiel 2.11 Datumszeile und Datum extrahieren.

```
dateline = ld.xpath(
    ".//tei:seg[@rend='dateline']",
    namespaces=ns
)

date_el = ld.xpath(
    ".//tei:date[1]",
    namespaces=ns
)

date_when = date_el[0].get("when") if date_el else None
```

Hier zeigt sich ein wichtiger Punkt: Nicht jedes Feld ist immer vorhanden. Fehlende Werte gehören zur Datenrealität antiker Überlieferung.

2.3.5 Absender und Empfänger als Heuristik

Oft steht in der Anrede beispielsweise:

Cicero Bruto salutem

oder

M. Cicero S.D. P. Lentulo procos.

Wir verwenden eine einfache Heuristik, um Absender und Empfänger zu trennen:

Code-Beispiel 2.12 Heuristische Extraktion von Absender und Empfänger.

```
def extract_sender_recipient_from_salute(salute):
    if not salute:
        return None, None

    s = salute.strip()

    m = re.search(r"\bS\s*\.\?\s*D\s*\.\?\b", s)
    if m:
        sender = s[:m.start()].strip()
        recipient = s[m.end():].strip()
        return sender, recipient

    parts = s.split()
    if len(parts) < 2:
        return None, None

    return parts[0], parts[1]
```

Dieses Verfahren funktioniert noch nicht fehlerfrei, aber fängt die meisten Fälle ab. Es ist danach an uns, die Daten auf Korrektheit zu überprüfen. Sie ist jedoch für den Beginn ausreichend für erste explorative Analysen. Für genaue Netzwerkvisualisierungen müssen wir diese Methode noch verfeinern.

2.3.6 Vom XML zur Tabelle

Die extrahierten Informationen werden in Dictionaries gespeichert und anschließend in einen **pandas**-DataFrame überführt:

Code-Beispiel 2.13 DataFrame aus Metadaten erzeugen.

```
import pandas as pd

df = pd.DataFrame(rows)
df.head()
```

Hier erfolgt der methodische Übergang:

hierarchisches XML -> flache Tabelle

Beide Repräsentationsformen sind legitim, dienen jedoch unterschiedlichen Analyseformen. Aber was genau ist `pandas`? `pandas` ist eine Python-Bibliothek zur Verarbeitung und Analyse strukturierter Daten. Sie wurde ursprünglich für statistische und ökonomische Daten entwickelt, wird heute aber in nahezu allen datenorientierten Disziplinen verwendet.

Das zentrale Objekt in `pandas` ist der sogenannte *DataFrame*. Ein *DataFrame* ist konzeptionell nichts anderes als eine Tabelle:

- Zeilen entsprechen Beobachtungen oder Einheiten,
- Spalten entsprechen Variablen oder Merkmalen.

In unserem Fall bedeutet das:

ein Brief = eine Zeile

Absender, Empfänger, Datum, Buchnummer usw. = Spalten

Warum brauchen wir `pandas`? Es gibt drei zentrale Gründe:

1. **Strukturierung** Wir transformieren hierarchisches XML in eine flache, analysierbare Tabellenstruktur.
2. **Filterung und Gruppierung** Wir können sehr einfach Fragen stellen wie:
 - Wie viele Briefe gibt es pro Korpus?
 - Wie viele Briefe sind datiert?
 - Welche Absender kommen am häufigsten vor?
3. **Statistische Grundoperationen** Mittelwerte, Häufigkeiten oder Anteile lassen sich mit wenigen Befehlen berechnen.

Ein DataFrame erzeugen Aus einer Liste von Dictionaries entsteht ein *DataFrame* mit einem einzigen Befehl:

Code-Beispiel 2.14 *DataFrame* aus einer Liste von Dictionaries erzeugen.

```
import pandas as pd

df = pd.DataFrame(rows)
df.head()
```

`df.head()` zeigt die ersten Zeilen der Tabelle an. Damit sehen wir sofort, ob unsere Extraktion funktioniert hat.

Typische Operationen mit pandas Oft wollen wir Objekte in einem DataFrame **gruppieren und zählen**. Dafür gibt es die `groupby`-Methode:

Code-Beispiel 2.15 Anzahl Briefe pro Korpus.

```
df.groupby("corpus").size()
```

Hier gruppieren wir alle Zeilen nach dem Feld `corpus` und zählen die Anzahl Briefe pro Gruppe.

Des Weiteren wollen wir oft einfache Statistiken wie **Anteile berechnen**. Dazu gibt es in `pandas` weitere integrierte Methoden wie `mean()`:

Code-Beispiel 2.16 Anteil datierter Briefe pro Korpus.

```
df["date_when"].notna().groupby(df["corpus"]).mean()
```

Da `True` intern als 1 und `False` als 0 behandelt wird, entspricht der Mittelwert dem Anteil datierter Briefe.

Nachdem uns die Daten als DataFrame vorliegen, können wir die Tabelle relativ simpel auf **fehlende Werte untersuchen**:

Code-Beispiel 2.17 Anteil fehlender Werte pro Feld.

```
df.isna().mean()
```

Hier zeigt sich unmittelbar, welche Felder stabil sind und welche oft fehlen.

Wir sehen, dass `pandas` nicht einfach ein technisches Hilfsmittel ist. Es zwingt uns zu einer Entscheidung:

Was ist unsere Analyseeinheit?

Indem wir sagen:

Ein Brief ist eine Zeile.

legen wir implizit fest, auf welcher Ebene wir argumentieren. Gleichzeitig verlieren wir dabei bestimmte Aspekte der hierarchischen Struktur des XML. Ein DataFrame ist flach. Ein TEI-Dokument ist verschachtelt.

Die Wahl des Datenmodells ist also nicht neutral, sondern bereits eine methodische Entscheidung. Für den Cicero-Briefwechsel ist die hierarchische Struktur nicht zentral, weshalb wir im Prinzip sagen können, dass die Repräsentation als XML gleich jener im DataFrame ist.

Fazit `pandas` ermöglicht es uns,

- strukturierte Tabellen zu erzeugen,
- schnell quantitative Fragen zu stellen,
- fehlende Daten sichtbar zu machen,
- und Ergebnisse reproduzierbar zu speichern.

Damit bildet es die zentrale Brücke zwischen Textstruktur und quantitativer Analyse.

2.3.7 Dataset exportieren

Code-Beispiel 2.18 Mini-Dataset als CSV speichern.

```
mini_cols = [  
    "corpus",  
    "book_n",  
    "letter_n",  
    "sender",  
    "recipient",  
    "date_when",  
    "year",  
    "dateline"  
]  
  
mini = df[mini_cols]  
mini.to_csv("outputs/cicero_letters_metadata_mini.csv", index=False)
```

Damit entsteht ein kleines, reproduzierbares Dataset, das unabhängig vom Notebook weiterverwendet werden kann.

2.3.8 Erste statistische Checks

Nun prüfen wir die Datenbasis kritisch:

Code-Beispiel 2.19 Briefe pro Korpus.

```
df.groupby("corpus").size()
```

Code-Beispiel 2.20 Anteil datierter Briefe pro Korpus.

```
df["date_when"].notna().groupby(df["corpus"]).mean()
```

Code-Beispiel 2.21 Anteil fehlender Werte.

```
mini.isna().mean()
```

Diese Schritte sind nicht bloss technische Spielerei, sondern Teil der historischen Datenkritik:

- Welche Felder sind stabil?
- Wo fehlen systematisch Informationen?
- Welche Analysen sind dadurch möglich oder eingeschränkt?

Hier greifen Whites *constraints and biases*. Unsere Datenbasis ist durch Überlieferung, Edition und Strukturierung geprägt. Digitale Methoden machen diese Struktur explizit sichtbar.

2.3.9 Visualisierung

Einfache Grafiken helfen, Strukturen schnell zu erkennen:

Code-Beispiel 2.22 Briefe pro Korpus visualisieren.

```
df.groupby("corpus").size().plot(kind="bar")
```

Visualisierungen sind Interpretationsangebote. Sie beruhen auf bestimmten Feldern. Wenn diese Felder unsicher sind, ist auch die Grafik nur so stabil wie ihre Datenbasis.

2.3.10 Fazit

In diesem Abschnitt hast du:

- TEI-XML strukturell analysiert
- zentrale Metadaten extrahiert
- ein reproduzierbares Mini-Dataset erstellt
- erste quantitative Checks durchgeführt

Damit ist die Grundlage für weiterführende Analysen wie Zeitverläufe, Netzwerke oder räumliche Auswertungen gelegt.

Diskurs und Pragmatik

Lernziele Session 3: Diskurs und Pragmatik

Verstehen Du verstehst, dass pragmatische und diskursive Phänomene (z.B. Bitten, Rechtfertigungen, Selbstpositionierungen, Nähe- und Distanzmarkierungen) nicht direkt “gegeben” sind, sondern durch Modellierungsentscheidungen in analysierbare Daten überführt werden müssen. Du kannst erklären, wie *close reading* als Grundlage für die Operationalisierung solcher Kategorien dient.

Analysieren Du kannst kommunikative Funktionen in Ciceros Briefen identifizieren und in strukturierte, digital auswertbare Annotationen oder Kategorien überführen. Dabei reflektierst du, welche Reduktionen, Vereinfachungen oder Verschiebungen durch diese Formalisierung entstehen.

Anwenden Du kannst einfache digitale Verfahren (z.B. Annotation, Metadaten-Filter, Häufigkeitsanalysen, Kollokationsanalysen oder Mustererkennung) einsetzen, um diskursive Strategien über mehrere Briefe hinweg sichtbar zu machen und mit Einzelanalysen zu verbinden.

Bewerten Du kannst beurteilen, inwiefern digitale Operationalisierungen pragmatischer Phänomene valide und reliabel sind und wo quantitative Ergebnisse durch Vorverarbeitung, Korpuszuschnitt oder Kategorisierung verzerrt werden.

Erzeugen Du kannst eine kleine, transparent dokumentierte digitale Analyse erstellen, in der du ein diskursives Muster (z.B. Veränderung von Ton, Rolleninszenierung oder Argumentationsstruktur) definierst, operationalisierst und auf Korpusebene überprüfst.

Nachdem wir in Kapitel 2 den Briefwechsel Ciceros in eine einheitliche Form

gebracht haben, ist er nun bereit für weitere Analysen. Bevor wir uns jedoch inhaltlich dem Inhalt zuwenden können, das heisst, der Struktur der eigentlichen Korrespondenz, sollten wir uns der Struktur des Textes *per se* zuwenden. In der Computerlinguistik ist der erste logische Schritt nach der Akquirierung des Textes das Splitting desselben in kleinere Teile. Dazu gehören in der Regel die Zerlegung des Inhalts in Sätze und darauf das Auftrennen der Sätze in einzelne Wörter, auch Tokenisierung genannt. Als Nächstes folgt die Wortartenannotation (EN *Part-of-Speech Tagging*) und das Parsing, welches die syntaktische Struktur eines Satzes markiert. Diese technischen Schritte erläutern wir im Unterkapitel 3.3.

Rein inhaltlich gesehen verweilen wir in diesem Kapitel auf der Brief- und Satzebene. Wir gehen deshalb nicht, was man wohl in einem Kurs in der Computerlinguistik machen würde, nicht stringent “bottom up” vor, sondern eher “top down.” Was können wir denn auf der Brief- und Satzebene bereits an Erkenntnissen gewinnen? Respektive: Welches Themenfeld widmet sich eher den “übergeordneten” Strukturen? Auf der einen Seite ist klar, die Diskursanalyse zu nennen. In der Computerlinguistik besteht der Diskurs aus einer Kette kohärenter, also inhaltlich zusammenhängender Sätze (Jurafsky & Martin, 2019). Diese Definition nehmen wir auch für das Unterkapitel 3.1 an, in welchem wir betrachten, was Diskurs weiter bedeutet und wie er sich in der Antike manifestierte.

Die Pragmatik ist ein eng mit dem Diskurs zusammenhängendes Teilfeld in der Linguistik, das als Forschungsgebiet die Sprecherintentionen in situativen Kontexten umfasst. Dieser Kontext kann sich als Diskurs auszeichnen, weshalb wir ihn wohl der Pragmatik gegenüber als übergeordnet ansehen können. Die genaue Differenzierung der beiden Teilgebiete ist und war jedoch immer wieder Gegenstand wissenschaftlicher Auseinandersetzungen (Brinton, 2005). Wir machen den Unterschied zwischen Diskurs und Pragmatik dennoch und geben dann in Kapitel 3.2 einige Inputs zu den Intentionen der Briefautor:innen im Cicero-Briefwechsel.

3.1 Diskurs

Das Gebiet der Diskursanalyse ist breit gefächert und reicht von Roman Jakobson, über Jürgen Habermas bis zu Michel Foucault, wobei wir mit Ersterem beginnen Habermas überspringen und bei Foucault enden möchten.

Warum mit Jakobson beginnen? Wir haben bis dato noch nicht wirklich etabliert, wie in Briefen kommuniziert wird. Da wir im Folgenden Prozesse automatisieren möchten, ist es ungünstig, wenn wir im luftleeren Raum operieren. Deshalb sollten wir uns einer Theorie bedienen, die erklärt, wie Briefe im Allgemeinen funktionieren. Dabei gibt es zig Kommunikationsmodelle, die dafür in Frage kommen. Wir nähern uns der Analyse der Korrespondenz jedoch von einer computerlinguistischen Perspektive, weshalb Modelle aus der Linguistik wohl passender sind als solche aus der Psychologie oder Soziologie.

Darum betrachten wir die von Jakobson 1960 formulierte Theorie der Funktionalität der Sprache im Allgemeinen. Abbildung 3.1–wie bei Shiffrin (2010,

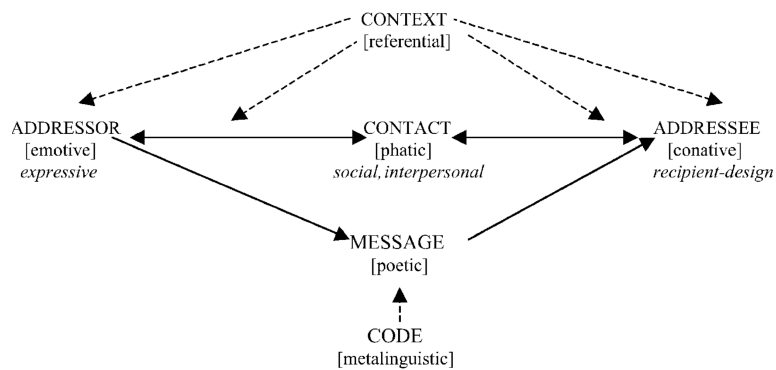


Abbildung 3.1 Sprechsituation und Sprachfunktionen nach Jakobson (2010) (wie publiziert bei Shiffrin, 2010, S.193). Jede Facette einer Situation erscheint in Grossbuchstaben; die Sprachfunktion befindet sich in eckigen Klammern.

S. 193) publiziert–stellt das Modell dar, das Sätze, die in einer Sprache geäußert werden, entsprechend einordnet. Dabei fällt uns sofort die Beziehung zwischen ADDRESSOR und ADDRESSE auf, die wir in einer Briefkorrespondenz mit ABSENDER und EMPFÄNGER gleichsetzen können. Zwischen den beiden Parteien steht der CONTACT oder die Kontaktebene. Diese kennzeichnet sich unter anderem durch *phatische* Sätze, welche die Beziehung von Absender und Empfänger genauer zum Ausdruck bringen. Auf dieser Ebene wird auch der Status der Parteien ersichtlich. In Briefwechseln ist der Kontakt kein Vis-à-Vis, sondern findet, wie wir schon gesehen haben, *in absentia* statt.

Die emotionale Funktion auf der Seite des Absenders wird durch den CONTEXT (Kontext) beeinflusst und gibt also sich darin befindliche Zustände oder interne Zustände wieder. Konative Funktionen beschreiben das Verhältnis zwischen Fakten aus dem Kontext und dem Empfänger oder die Interpretation einer Aussage des Absenders auf sozialer Ebene. Die MESSAGE oder die Nachricht gelangt vom Absender zum Empfänger (da sich Jakobson damit beschäftigte, was eine Äusserung poetisch macht, finden wir hier die poetische Facette). Entscheidend ist, dass diese Nachricht vom Kontext geprägt ist und einem gemeinsamen CODE (Kode) folgt, damit die Nachricht für den Empfänger verständlich ist. Das heisst, dass die Nachricht beim Empfänger, vom Kontext beeinflusst, enkodiert und beim Empfänger entsprechend dekodiert wird.

Diskurs findet gemäss Shiffrin (2010) auf verschiedenen Ebenen statt. Besonders für Briefwechsel scheint dies zentral zu sein. Es ist beispielsweise entscheidend, wie sich das Teilnehmerfeld des Diskurses zusammensetzt. Wer sind die Akteure? Wer gestaltet den Diskurs mit? Bei Korrespondenz denkt man unweigerlich direkt an Absender und Empfänger. Aber sicher auch die darin erwähnten Personen tragen ihren Teil zum Diskurs bei. Des Weiteren spielt die Struktur des Austauschs eine Rolle. In Briefen ist dies relativ gut geregelt: Person A schickt Person B einen Brief. Daraufhin antwortet Person B Person A mit einem weiteren Brief. Das kann ins Unendliche so weitergehen. Aber es sind auch abweichende Modelle möglich: Zum Beispiel könnte Person B den Brief von Person A an Person C weiterleiten. Diese Person C, wenn nicht schon Teil des Diskurses, wird dann neu Teil davon. Zudem können Nachrichten, oder anders gesagt, die in Briefen enthaltenen Sprechakte, Handlungen beim Gegenüber

auslösen und so den Diskurs beeinflussen. Die Ebene des Informationsstatus beschreibt, wie sich die Informationen über den Diskurs unter den involvierten Personen verteilen. In Briefwechseln hat dies mit Sicherheit einen Einfluss auf das Verfassen eines Briefes. Der Absender muss den Informationsstand des zukünftigen Empfängers abschätzen können. Und zuguterletzt muss der Verfasser sich noch Gedanken darüber machen, wie er die Informationen mitteilen und strukturieren möchte.

Obwohl uns, wie wir in Kapitel 2.1 gesehen haben, nur die Hälfte des Dialogs im Briefwechsel vorliegt, können wir damit dennoch auf die Diskursebene schliessen. Wir dürfen Dialog nämlich nicht mit Diskurs gleichsetzen. Bei der Analyse des Diskurses ist es nun aber trotzdem wichtig, dass wir das Kommunikationsmodell von Jakobson vor Augen haben. Denn wenden wir dieses auf den Ciceronianischen Briefwechsel an, merken wir, dass der Kontext hauptsächlich von Cicero geprägt wird, respektive, erfahren und sehen wir ihn durch Cicero. Und nicht nur das: auch die Mitteilungen und demzufolge auch der Kode stammen hauptsächlich aus Ciceros Feder. Das müssen wir bei der Analyse des Diskurses, der aus den einzelnen Sätzen der Korrespondenz besteht, berücksichtigen.

Denn wie die Sätze in einer Sprache zum Zwecke der Kommunikation strukturiert sind, bringt uns zum einen zur Frage, wie sich der Diskurs zusammensetzt, und zum anderen: Warum denn Foucault? Im Jahre 1970 hielt er seine Antrittsvorlesung zum Thema “Die Ordnung des Diskurses” (Foucault, 1991). Dabei stellt für Foucault der Diskurs nicht einfach eine sprachliche Ordnung dar, sondern er definiert sich durch eine geordnete Praxis.

Es sind also gewisse Bedingungen im Spiel. Beim Ausschluss von aussen regeln Verbote den Diskurs: Nicht alles ist sagbar. Nicht jede Person darf alles sagen. Nicht jede Aussage gilt als wahr. Für Cicero konnte das Beispiel heissen, dass sich politische Kritik innerhalb legitimer Grenzen bewegen musste. Zudem wurde von aussen durch gewisse Autoritäten bestimmt, welche Aussagen zirkulieren dürfen. Das führte dazu, dass die Wahrheit institutionell eingebettet und definiert ist. Hierbei sind die Briefe Teil dieser Ordnung. Sie sind zwar private Kommunikation, aber sie formten den Diskurs innerhalb eines politischen Machtfeldes entscheidend mit.

Diskurse sind aber auch intern geordnet. So ermöglicht der Kommentar in gewisser Weise das ständige Aufkommen des Diskurses, da immer wieder auf ihn Bezug genommen wird. Der Autor reguliert den Diskurs, indem er ihn startet, gruppiert und zusammenhält. Die Disziplin gibt hierbei vor, nach welchen Regeln, welche mit der Zeit etabliert worden sind, der Diskurs (theoretisch bis ins Unendliche) weitergeführt werden kann.

Zuguterletzt ist der Zugang zum Diskurs begrenzt. Dies erfolgt durch das Ritual. Wird beispielsweise ein unqualifizierter Beitrag zum Diskurs geleistet, wird die Person, von der dieser stammt, vom Diskurs ausgeschlossen. Es bilden sich so Diskursgesellschaften, welche den Diskurs pflegen und aufrechterhalten (mithilfe der Disziplin). Die Doktrin sagt dann schliesslich, welche Aussagentypen zulässig sind, beschränkt also in gewisser Weise den Diskurs (aber nicht so sehr, dass er zum Erliegen kommt).

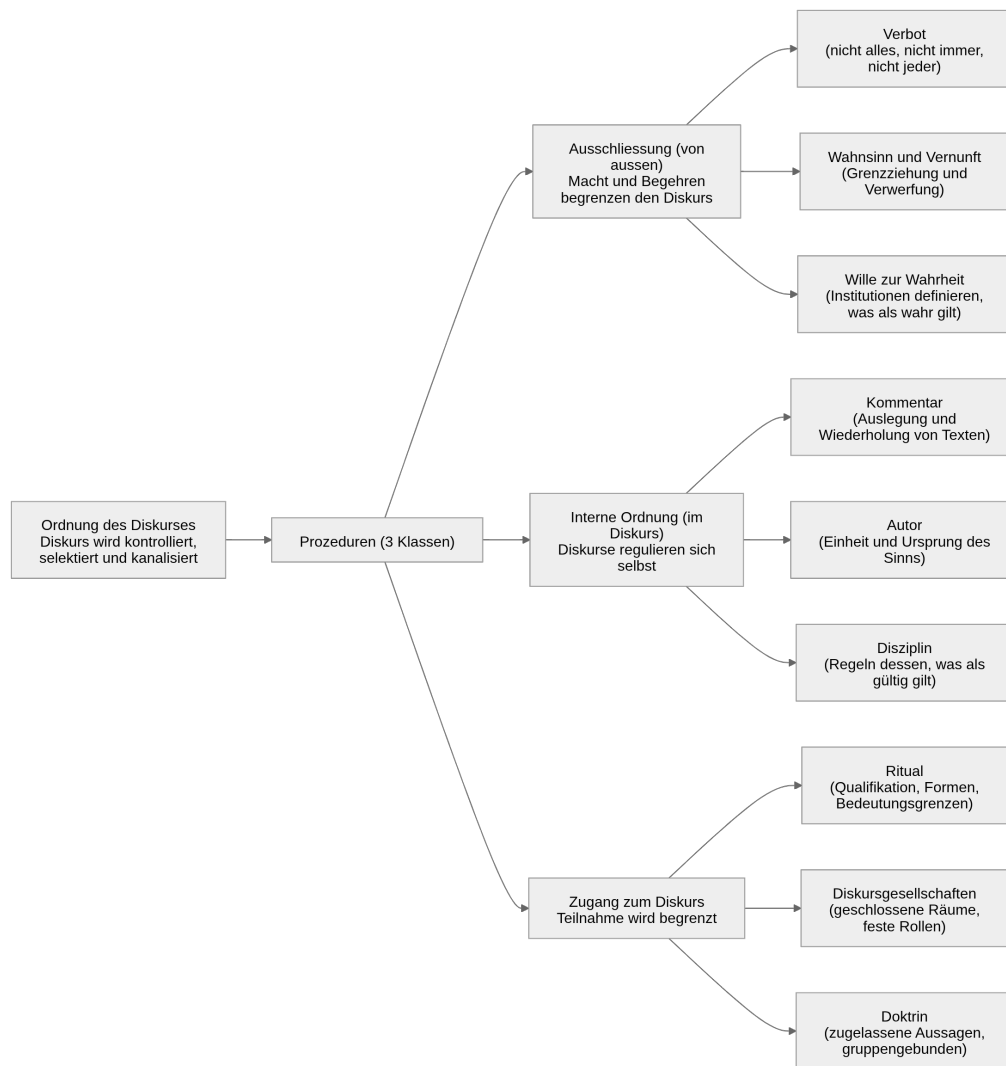


Abbildung 3.2 Ordnung des Diskurses nach Foucault (1991). Diskurse werden durch drei Klassen von Prozeduren strukturiert: durch Ausschliessung von aussen, durch interne Ordnungsmechanismen sowie durch die Begrenzung des Zugangs zum Sprechen. Diese Mechanismen bestimmen, was sagbar ist, wer sprechen darf und was als wahr gilt.

Es ist für die interne Ordnung und den Zugang zum Diskurs relativ klar, wie dies im Briefwechsel stattfinden muss. Wir können auch Theorien aufstellen, wie das in der Cicero-Korrespondenz stattfand. So wird Cicero als Autor, der sehr viel geschrieben hat (und von dem am meisten überliefert ist), viel zur Diskursgestaltung beigetragen haben. Gleichzeitig können wir auch die Diskursgesellschaft als solches nachzeichnen. Sie besteht direkt aus Cicero und seinen Briefpartner:innen sowie wahrscheinlich anderen Personen aus dem direkten Umfeld, die teilweise auch in den Briefen erwähnt werden.

Bei all der Liebe zu Modellen müssen wir bei Foucault trotzdem beachten, dass ihn besonders interessierte, wie sich Diskurse auf Machtgefüge auswirkten. Das wird aber auch für Cicero wieder zentral, wenn wir uns in Kapitel ?? genauer der Machtausübung Ciceros zuwenden.

Den “Machtdiskurs” lassen wir also für den Moment aussen vor. Es gilt aber trotzdem noch zu betonen, dass bereits verschiedene Diskurse im Bereich der Antike untersucht worden sind und sich Foucaults Modell immer ein wenig anders auf diese anwenden lässt. So widmen sich beispielsweise Eickhoff et al.

(2016) dem Mussediskurs in antiken Briefen und gehen der Frage nach, wieso gewisse Autoren die Musse als optimale Voraussetzung des Lernens betrachteten und warum beispielsweise Cicero immer dann die Vorteile eines ruhigen Lebens ausserhalb des Politzirkuses der Republik erörtere, wenn er sich gerade wieder im Exil befand. Hier rücken Cicero und seine Gefühlswelt definitiv in den Mittelpunkt des Diskurses, und die Diskursgesellschaft wird sich wohl in einen sehr engen Kreis fassen lassen.

Bei White zeigen sich Jakobson und Foucault für die Definition des Diskurses vereinigt:

That a letter is itself the transcript of a discourse is a further consequence of writing. Every letter writer by that act creates and entrusts to someone else a material record of a sort that is rarely obtained for conversations, even in our era when sound recordings are technically possible. A third way in which an exchange of letters differs from a conversation is that participation is typically more restricted. In a strict sense, of course, only one party to a correspondence is active at any moment [...]. (White, 2010, S. 65)

Das Medium der Schriftlichkeit und das, was darin enkodiert werden kann, gehen direkt auf Jakobsons Kode zurück. Der Aspekt des Anvertrauens von Material geht auf Shiffrins Ebenen des Informationsstatus und der Informationsvermittlung zurück. Gleichzeitig ist die Restriktion des Teilnehmerkreises ein Ausdruck von Foucaults Ordnung des Diskurses, und zwar im Bereich “Zugang zum Diskurs” und der Bildung von Diskursgesellschaften.

3.2 Pragmatik

Unterka-
pitel zu
Pragma-
tik schrei-
ben.

3.3 Textsegmentierung mit *spaCy*

Nachdem wir in Session 2 die TEI-XML-Datenstruktur in ein einfacheres Datenset mit Metadaten pro Brief umgewandelt und als `cicero_letters.csv` exportiert haben, können wir nun mit der sprachlichen Vorverarbeitung beginnen. Für viele digitale Analysen (z.B. Diskursmarker, syntaktische Muster, Rollen- und Argumentationsstrukturen) ist es hilfreich, den Text in kleinere Einheiten zu zerlegen und linguistisch zu annotieren. Das dazugehörige Jupyter Notebook befindet in der [JupyterLite-Umgebung](#).

3.3.1 Ziel dieses Abschnitts

Wir führen drei Standard-Schritte aus der Computerlinguistik ein:

1. **Satzsegmentierung:** Ein Text wird in Sätze zerlegt.
2. **Tokenisierung:** Sätze werden in Tokens (Wortformen, Satzzeichen) zerlegt.

3. **Annotation:** Tokens erhalten Labels (z.B. Wortart/POS) und werden syntaktisch verknüpft (Dependenz-Parsing).

Wir verwenden dafür *LatinCy*¹ innerhalb des *spaCy*-Universums (Burns, 2023). Optional kann auch das sogenannte *Classical Language Toolkit*² (CLTK) (Johnson et al., 2021) dafür eingesetzt werden,³ um eine Pipeline für Latein zu laden. Die Funktionalitäten und Auszeichnungen sind leicht anders als bei *LatinCy*. Da Installationen je nach Umgebung unterschiedlich sind, ist der Code so gestaltet, dass er Fallbacks besitzt.

3.3.2 Briefdaten einlesen

Das Daten-CSV enthält pro Brief eine Zeile. Wir lesen es als *DataFrame* ein:

Code-Beispiel 3.1 Daten-CSV einlesen.

```
from pathlib import Path
import pandas as pd

csv_path = Path("outputs") / "cicero_letters.csv"
df = pd.read_csv(csv_path)
df.head()
```

3.3.3 *LatinCy* initialisieren

Wir laden eine Pipeline *nlp*. Falls das *LatinCy*-Modell verfügbar ist, verwenden wir dieses bevorzugt. Ansonsten wird ein *spaCy*-Lateinmodell geladen. Falls kein Modell verfügbar ist, nutzen wir *spacy.blank("la")*; dann funktionieren Tokenisierung und Satzsegmentierung, aber keine POS/Parsing-Annotation.

Code-Beispiel 3.2 Pipeline laden (mit Fallbacks).

```
import spacy

def load_nlp():
    try:
        return spacy.load("la_core_web_lg")
    except Exception:
        print("No LatinCy model, initialising blank model")
        nlp = spacy.blank("la")
        if "sentencizer" not in nlp.pipe_names:
            nlp.add_pipe("sentencizer")
        return nlp

nlp = load_nlp()
nlp.pipe_names
```

¹Siehe <https://spacy.io/universe/project/latincy>.

²Siehe <https://cltk.org>.

³Hier nicht demonstriert.

Mit `nlp.pipe_names` erhalten wir eine Übersicht der Verarbeitungsschritte, welche in der Pipeline ausgeführt werden. Für LatinCy sieht diese folgendermaßen aus: `['senter', 'normer', 'tok2vec', 'tagger', 'morphologizer', 'trainable_lemmatizer', 'parser', 'lookup_lemmatizer', 'ner', 'remorpher']`

Das heisst, dass nacheinander die eben aufgeführten Pipeline-Elemente aufgerufen werden. Beispielsweise teilt `senter` ein Dokument in einzelne Sätze. Der `tagger` ist anschliessend für die Tokenisierung und die Wortartenannotation zuständig. Alle Informationen werden in eine `spaCy Doc`-Klasse überführt, aus der die einzelnen Ebenen und Annotationen abgerufen werden können.

3.3.4 Ein Brief als Beispiel

Wir analysieren einen Brief und schauen uns Tokens, POS und Abhängigkeiten an:

Code-Beispiel 3.3 Tokens, POS und Abhängigkeiten inspizieren.

```
doc = nlp(df.loc[0, "text"])
[(t.text, t.lemma_, t.pos_, t.dep_) for t in doc[:25]]
```

Das Resultat ist hier eine Liste bestehend aus der Oberflächenform des Wortes, dessen Lemma, der Wortart und der Abhängigkeitsrelation. Konkret sieht ein Teil des Outputs dafür so aus:

```
('petitionis', 'petitio', 'NOUN', 'ROOT'),
('nostrae', 'noster', 'DET', 'det'),
(',', ' ', 'PUNCT', 'punct'),
('quam', 'qui', 'PRON', 'obj'),
('tibi', 'tu', 'PRON', 'obl:arg'),
('summae', 'superus', 'ADJ', 'amod'),
('curae', 'cura', 'NOUN', 'ccomp'),
('esse', 'sum', 'AUX', 'cop'),
('scio', 'scio', 'VERB', 'acl:relcl'),
(',', ' ', 'PUNCT', 'punct'),
('huius', 'hic', 'DET', 'det'),
[...]
```

Wir sehen hier der Reihe nach pro Zeile die von LatinCy hinzugefügten Informationen zu den einzelnen Tokens. So finden wir für das zweite Wort *nostrae* (DE *unser*) korrekterweise das Lemma *noster*. An dritter Stelle folgt die Wortart, annotiert nach den sogenannten *Universal Dependencies* (UD).⁴ An letzter Stelle folgt noch die Abhängigkeitsinformation, das heisst, die Relation des Wortes zu seinem entsprechenden head. Würde man diesen für *nostrae* mit `t.head` ausgeben, erhielte man *petitionis*.

Optional kann die syntaktische Struktur in Jupyter mit `displacy` visualisiert werden. Abbildung 3.3 zeigt den Abhängigkeitsbaum des ersten Satzes aus dem eben analysierten Brief.

⁴Siehe <https://universaldependencies.org>. Die UD haben zum Ziel, ein Standard-Tagset bereitzustellen, das für sämtliche Sprachen verwendet werden kann. Siehe im Gegensatz dazu zum Beispiel das *Stuttgart-Tübingen Tagset* (STTS).

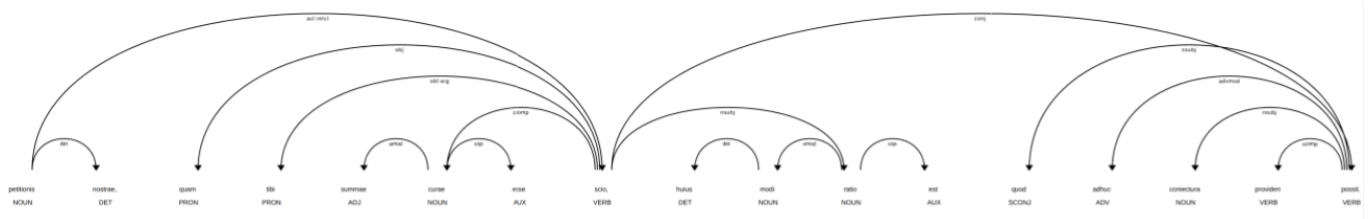


Abbildung 3.3 Dependenz-Baum des ersten Satzes.

Parsing-Strukturen können beim Annotieren und Erkennen von Diskurselementen hilfreich sein, wie das beispielsweise für die *Rhetorical Structure Theory* nötig ist (siehe Unterkapitel 3.4).

3.3.5 POS-Verteilungen (gesamt und nach Subkorpus)

Um stilistische Unterschiede zwischen Teilkorpora sichtbar zu machen, zählen wir POS-Tags in einer Stichprobe und vergleichen sie nach corpus:

Code-Beispiel 3.4 POS zählen und vergleichen.

```
def pos_counts(texts):
    counts = {}
    for doc in nlp.pipe(texts, batch_size=20):
        for tok in doc:
            if tok.is_space:
                continue
            pos = tok.pos_ if tok.pos_ else "?"
            counts[pos] = counts.get(pos, 0) + 1
    return counts

sample = df.sample(n=min(200, len(df)), random_state=42)
pos_total = pos_counts(sample["text"].tolist())
```

Die Ergebnisse lassen sich anschliessend als Balkendiagramm darstellen. Wichtig ist dabei stets die Datenkritik: Wenn ein Modell schlecht segmentiert oder falsch taggt, dann sind auch die Visualisierungen nur so stabil wie diese Vorverarbeitung.

3.3.6 Fazit

Mit *LatinCy* verfügen wir über eine Pipeline, die aus Brieftexten linguistisch annotierte Daten generiert. Damit können wir in den nächsten Schritten diskursive Muster operationalisieren, z.B. Diskursmarker, syntaktische Konstruktionen oder wiederkehrende Argumentationsschemata.

3.4 Diskurskohärenz

Wenn White betont, dass ein Brief ein Transkript für den Diskurs ist (White, 2010, S. 65), stellt sich die Frage, wie dieser Diskurs intern organisiert ist. Die

Computerlinguistik beschreibt Diskurse als hierarchisch strukturierte Einheiten, deren Teile durch semantische Relationen miteinander verbunden sind (Jurafsky & Martin, 2019). Dependenz-Analysen, so wie wir sie im vorhergehenden Unterkapitel erzeugt haben, können erste Hinweise liefern, da bedeutungstragende Einheiten selten syntaktische Grenzen “sprengen”. Um die Diskursebene zu beleuchten und greifbar zu machen, müssen diese Strukturen nun jedoch in Zusammenhang gebracht werden. Wir wollen dies, teilgestützt Jurafsky und Martin (2019), mit einem theoretischen Fundament tun, das wir im Folgenden erläutern.

3.4.1 Rhetorical Structure Theory

Ein einflussreiches Modell hierfür ist die *Rhetorical Structure Theory* (RST) (Mann & Thompson, 1988). RST geht davon aus, dass Texte aus Einheiten bestehen, die in funktionalen Beziehungen zueinander stehen. Typischerweise wird zwischen einem *Nucleus* (zentraler Aussagekern) und einem *Satellite* (unterstützende Einheit) unterschieden. Der *Satellite* erfüllt dabei eine bestimmte Funktion, etwa Begründung, Erläuterung oder Kontrast. Nachfolgend seien einige solcher Relationen wiedergegeben. Wir möchten jedoch darauf hinweisen, dass es noch Relationen gibt.

Begründungsrelation (Reason) Att. 2,6, mit welchem auch White (2010) sein erstes Kapitel startet, beginnt mit einer solchen Relation:

A scribendo prorsus abhorret animus. etenim quae constitueram magnum opus est.

“Vom Schreiben hat mein Geist völlig Abstand genommen. Denn was ich mir vorgenommen hatte, ist ein grosses Werk.”

Der erste Satz bildet den *Nucleus*: Cicero erklärt seine Schreibunlust. Der zweite Satz fungiert als *Satellite* mit der Relation *Reason*: Er begründet diese Aussage.

Nach RST stabilisiert der *Satellite* die Interpretation des *Nucleus*. Hier dient die Begründung zugleich (ganz im Sinne von White) der sozialen Legitimation gegenüber Atticus. Die Kohärenz ist also sowohl rhetorisch als auch sozial funktional.

Elaboration und argumentative Verdichtung Cicero führt seine Begründung weiter aus:

ita valde Eratosthenes, quem mihi proposueram, a Serapione et ab Hipparcho reprehenditur. quid censes si Tyrannio accesserit? et hercule sunt res difficiles ad explicandum.

“Eratosthenes, den ich mir zum Vorbild genommen hatte, wird von Serapion und Hipparch stark kritisiert. Was meinst du, wenn nun auch Tyrannio hinzukommt? Und wahrlich, die Dinge sind schwierig zu erklären.”

Hier entfaltet sich eine Serie von *Elaboration*-Relationen: Die Schwierigkeit des Vorhabens wird weiter präzisiert und argumentativ verdichtet.

RST würde diese Passage als eine hierarchische Struktur beschreiben: Die ursprüngliche Begründung (*magnum opus est*) wird durch zusätzliche Satelliten ausgebaut. Der Diskurs wird dadurch kohärent erweitert.

Thematischer Wechsel als Diskursmarker Später markiert Cicero explizit einen Wechsel:

Sed ut ad rem, scripsi ad quaestores urbanos de Quinti fratris negotio.

“Doch um zur Sache zu kommen: Ich habe den städtischen Quaestoren wegen der Angelegenheit meines Bruders Quintus geschrieben.”

Sed ut ad rem fungiert als Diskursmarker. In RST-Termini signalisiert dieser Ausdruck den Übergang zu einem neuen Nucleus. Die vorangehende Passage über *otium* und Schreibunlust wird abgeschlossen; ein neuer thematischer Block beginnt. Solche Marker sind metadiskursiv. Im Sinne Jakobsons betreffen sie die metasprachliche Funktion: Der Sprecher kommentiert die Struktur seiner eigenen Äußerung.

Kontrast und Selbstpositionierung Besonders deutlich wird die diskursive Positionierung in folgender Passage:

esse locum tam prope Romam ubi multi sint qui Vatinius numquam viderint, ubi nemo sit praeter me qui quemquam ex viginti viris vivum et salvum velit, ubi me interpellat nemo, diligant omnes!

“Es gibt einen Ort so nahe bei Rom, wo viele Vatinius nie gesehen haben, wo ausser mir niemand wünscht, dass einer der Zwanzigmänner heil und lebendig sei, wo mich niemand unterbricht und alle mich mögen!”

Hier entsteht Kohärenz durch eine parallele syntaktische Struktur (*ubi ... ubi ... ubi ...*). Diese Reihung bildet eine Serie von *Elaboration*-Satelliten, die das Bild des idealen Rückzugsortes ausmalen. Gleichzeitig ist die Passage stark evaluativ. Nach White konstituiert Cicero hier eine soziale Position: Er konstruiert ein Gegenbild zu Rom und inszeniert sich selbst innerhalb dieser Opposition.

3.4.2 Phatische Stabilisierung des Kontakts

Am Ende des Briefes heisst es:

Aliud quid? etiam: quando te proficisci istinc putes fac ut sciam.

“Sonst noch etwas? Auch dies: Lass mich wissen, wann du von dort aufzubrechen gedenkst.”

Diese Passage erfüllt eine phatische Funktion im Sinne Jakobsons: Sie stabilisiert den Kontakt zwischen Absender und Empfänger.

RST würde hier keine starke argumentative Relation annehmen; vielmehr handelt es sich um eine neue Diskurseinheit mit eigener Funktion. Der Diskurs wird nicht beendet, sondern auf Fortsetzung angelegt. Genau dies betont White: Briefe halten Beziehungen aktiv.

3.4.3 Kohärenz als soziale Praxis

Die zunächst oberflächliche Analyse von *Att.* 2,6 zeigt, dass Diskurskohärenz auf mehreren Ebenen entsteht:

- durch explizite Begründungen (*etenim*),
- durch elaborierende Erweiterungen,
- durch Diskursmarker (*sed ut ad rem*),
- durch parallele syntaktische Strukturen,
- durch phatische Kontaktaufnahme.

In der RST lassen sich diese Elemente als funktionale Relationen zwischen Nucleus und Satellite modellieren. Doch wie White deutlich macht, ist diese Struktur nicht bloss formal. Der Brief entsteht unter Bedingungen von Distanz, eingeschränkter Teilnahme und sozialer Erwartung. Kohärenz dient daher nicht nur der Verständlichkeit, sondern der Beziehungspflege, Legitimation und Selbstpositionierung.

Der Brief des Trebonius (*Fam.* 12,16) enthält weitere Relationen, die sich präzise anhand der RST modellieren lassen.

Evidenz (Evidence) Eine klare *Evidence*-Relation findet sich in der Begründung von Trebonius' Freude:

qua ex re quantam voluptatem ceperim scire potes etiam me tacente. non enim nescis quanti te faciam.

“Aus dieser Sache kannst du erkennen, wie grosse Freude ich empfand, selbst wenn ich schweige. Denn du weisst ja, wie sehr ich dich schätze.”

Die Wertschätzung Ciceros dient hier als Beleg für die behauptete Freude. Der zweite Satz stützt die emotionale Aussage des ersten. Der Satellite liefert also nicht bloss zusätzliche Information, sondern Evidenz für die Aufrichtigkeit des Gesagten.

Attribution Eine weitere Relation ist *Attribution*:

audiebam quaedam turbulenta.

“Ich hörte von gewissen Unruhen.”

Hier wird Information ausdrücklich einer externen Quelle zugeschrieben. Die Aussage über politische Turbulenzen ist epistemisch markiert und damit als vermittelte Information gekennzeichnet. Solche Attributionen strukturieren Diskurse, indem sie Verantwortlichkeiten für Wissen verteilen. In diesem Fall ist der Nucleus “leer,” also nicht spezifiziert.

Liste (List, multinukleare Relation) Besonders deutlich tritt eine *List*-Relation in folgender Passage hervor:

non enim nescis quanti te faciam et quam pro nostro veterrimo verissimo-que amore omnibus tuis etiam minimis commodis, non modo tanto bono, gaudeam.

“Du weisst ja, wie sehr ich dich schätze und wie ich mich aufgrund unserer alten und aufrichtigen Freundschaft über alle deine Vorteile freue, nicht nur über einen so grossen.”

Hier werden mehrere gleichrangige Gründe nebeneinandergestellt: Wertschätzung, alte Freundschaft, Freude über jedes Gute. Diese Einheiten stehen nicht in einem hierarchischen Verhältnis, sondern bilden eine koordinierte Struktur. In RST handelt es sich um eine multinukleare Relation vom Typ *List*. Solche Strukturen erzeugen Kohärenz durch additive Verdichtung. Gleichzeitig verstärken sie die soziale Dimension des Briefes: Beziehung wird durch Aufzählung stabilisiert und intensiviert.

3.4.4 Elementary Discourse Units (EDUs)

Bevor Relationen wie *Reason*, *Evidence* oder *List* annotiert werden können, muss ein Text in sogenannte *Elementary Discourse Units* (EDUs) segmentiert werden. EDUs sind die kleinsten diskursrelevanten Einheiten eines Textes. In der Regel entsprechen sie unabhängigen oder eingebetteten Klauseln, nicht jedoch einzelnen Wörtern oder rein syntaktischen Phrasen.

Die Segmentierung in EDUs ist ein entscheidender Schritt, da RST-Relationen stets zwischen solchen Einheiten etabliert werden (Jurafsky & Martin, 2019). Ohne saubere Segmentierung gibt es keine modellierbare Kohärenzstruktur.

Ein Beispiel aus *Fam.* 12,16:

*Athenas veni a. d. XI Kal. Iun.
atque ibi vidi filium tuum deditum optimis studiis.*

Hier liegen zwei EDUs vor:

1. “Athenas veni ...”(Ankunft)
2. “atque ibi vidi filium tuum ...“ (Beobachtung)

Erst auf dieser Ebene kann die Relation *Background* identifiziert werden. Komplexer wird die Segmentierung bei hypotaktischen Strukturen. Etwa:

Qui cum mihi in sermone iniecisset se velle Asiam visere, non modo invitatus sed etiam rogatus est a me.

“Als er im Gespräch erwähnte, er wolle Asien besuchen, wurde er von mir nicht nur eingeladen, sondern sogar gedrängt.”

Hier lassen sich mindestens zwei EDUs unterscheiden:

1. “Qui cum .. Asiam visere” (Einbettung: Wunsch des Sohnes)

2. “non modo invitatus ... rogatus est a me” (Reaktion des Trebonius)

Die erste Einheit fungiert als Voraussetzung oder Auslöser der zweiten. Die Relation wäre hier als *Cause* oder *Condition* interpretierbar. EDU-Segmentierung ist nicht identisch mit der Satzsegmentierung. Ein einzelner lateinischer Satz kann mehrere EDUs enthalten, insbesondere bei Partizipialkonstruktionen, AcI-Strukturen oder kausalen Nebensätzen. Für die digitale Analyse bedeutet dies: Tokenisierung, POS-Tagging und Parsing liefern syntaktische Informationen, doch Diskurssegmentierung ist ein zusätzlicher Modellierungsschritt. Sie beruht auf interpretativen Entscheidungen.

Ein Brief ist also nicht einfach eine lineare Satzfolge, sondern eine strukturierte Kommunikationshandlung. Die Entscheidung, wo eine diskursrelevante Einheit beginnt oder endet, beeinflusst direkt, wie wir Kohärenz rekonstruieren. EDUs bilden somit die Brücke zwischen der formalen Sprachstruktur (Syntax) und der sozialen Praxis (Diskurs).

3.4.5 EDU-Segmentierung für RST-Parsing

Bevor ein Diskursbaum gebaut werden kann, muss der Text in sogenannte *Elementary Discourse Units* (EDUs) segmentiert werden. Jurafsky und Martin betonen: RST-Parsing geschieht in zwei Schritten. Der erste Schritt ist die Identifikation der Grenzen dieser EDUs. Eine EDU entspricht in der Regel einer selbständigen oder eingebetteten Klausel. Wichtig ist: Sie ist eine diskursrelevante Einheit, nicht bloss ein syntaktisches Fragment. Wir betrachten dazu oben schon erwähntes Beispiel aus *Fam.* 12,16.

3.4.6 RST-Parsing (vereinfacht)

Nachdem EDUs identifiziert wurden, beginnt der zweite Schritt: der Aufbau der Diskursstruktur. RST-Parsing bedeutet, dass EDUs schrittweise zu grösseren Einheiten kombiniert werden.

Vereinfacht funktioniert das so:

1. Eine EDU wird auf einen *Stack* gelegt (*Shift*).
2. Eine weitere EDU wird hinzugefügt (*Shift*).
3. Zwei EDUs werden zu einer Struktur verbunden (*Reduce*).

Die Verbindung erhält dabei:

- ein Relationslabel (z.B. Reason, Evidence, Elaboration, etc.),
- eine Nuklearitätsangabe (Nucleus, Satellite oder Joint).

Betrachten wir folgendes Beispiel aus dem Brief:

*qua ex re quantam voluptatem ceperim scire potes etiam me tacente.
non enim nescis quanti te faciam.*

Segmentiert in zwei EDUs:

1. “Ich habe grosse Freude empfunden.”
2. “Denn du weisst ja, wie sehr ich dich schätze.”

Parsing-Schritte (vereinfacht):

- Shift EDU 1
- Shift EDU 2
- Reduce(Reason, S<->N)

Das Resultat: EDU 1 ist der Nucleus, EDU 2 ist der Satellite mit der Relation *Reason*. Ein Beispiel für eine multinukleare Struktur findet sich hier:

non enim nescis quanti te faciam et ... gaudeam.

Hier stehen zwei Aussagen gleichrangig nebeneinander. Das Parsing würde eine Relation vom Typ *Joint* oder *List* erzeugen.

RST-Parsing ist im Kern nichts anderes als:

1. Text in diskursive Einheiten zerlegen.
2. Diese Einheiten schrittweise verbinden.
3. Jede Verbindung funktional benennen.

Moderne Systeme verwenden neuronale Modelle und Encoder-Decoder-Architekturen, doch konzeptionell bleibt das Prinzip identisch: Man sagt vorher, welche EDUs es gibt, welche EDU mit welcher anderen verbunden wird und welche Funktion sie haben. Gerade bei antiken Briefen ist dies besonders interessant, weil wir so sichtbar machen können, wie Argumentation, Beziehungspflege und Selbstpositionierung strukturiert sind. Die Abbildung 3.4 stellt einen Teil des Diskursbaumes aus *Fam.* 12,16 dar.

3.4.7 Digitale Diskursanalyse

Digitale Diskursanalyse kann solche Relationen annotieren und über mehrere Briefe hinweg vergleichen. Ihre historische Interpretation bleibt jedoch auf die Verbindung von struktureller Analyse und sozialer Kontextualisierung angewiesen. Für moderne Sprachen gibt es bereits sogenannte Diskursbaumbanken. Diese sucht man für das Lateinische jedoch vergeblich. Das heisst, dass wir, um im Anschluss erfolgreich automatische Diskursannotationssysteme bauen zu können, zuerst solche Baumbanken erstellen müssten.

Ein Weg dahin führt über die klassische manuelle Annotation. Hierbei bedient man sich üblicherweise einer bereits bestehenden Annotationssoftware wie Label Studio⁵ oder Prodigy.⁶ Die Funktionsweise ist für beide Frameworks identisch:

⁵Siehe <https://labelstud.io>.

⁶Siehe <https://prodi.gy>.

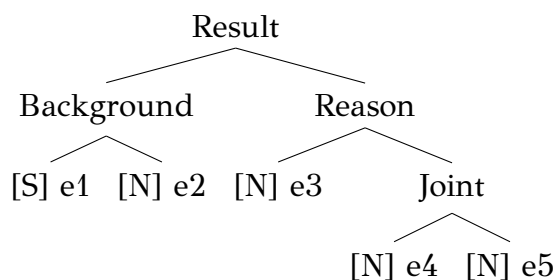


Abbildung 3.4 RST-Diskursbaum zu *Fam.* 12,16. [S] = Satellite, [N] = Nucleus. Joint = multinukleare Relation (gleichrangige Einheiten).

Verwendete Passagen (EDUs):

- e1** *Athenas veni a. d. XI Kal. Iun.*
 “Ich kam am 22. Mai nach Athen.”
- e2** *atque ibi ... vidi filium tuum deditum optimis studiis summaque modestiae fama.*
 “Und dort sah ich deinen Sohn, den besten Studien hingegeben und im Ruf grösster Anständigkeit.”
- e3** *qua ex re quantam voluptatem ceperim scire potes etiam me tacente.*
 “Daran kannst du erkennen, wie grosse Freude ich empfand, selbst wenn ich schweige.”
- e4** *non enim nescis quanti te faciam*
 “Denn du weisst ja, wie hoch ich dich schätze,”
- e5** *et quam ... amore ... omnibus tuis ... commodis ... gaudeam.*
 “und wie sehr ich mich aufgrund unserer alten und aufrichtigen Freundschaft über alles freue, was dir zugutekommt.”

Kurzinterpretation: *e1* liefert den situativen Rahmen (*Background*) für *e2* (Beobachtung über Marcus). *e3* ist die evaluative Reaktion (Freude) auf *e2* und wird durch *e4* und *e5* gleichrangig begründet (*Reason* + *Joint/List*). Der Brief stellt so Beziehungspflege und Legitimation als kohärente Diskursstruktur her.

Man lädt die zu annotierenden Daten hoch, definiert eine Annotationsumgebung und annotiert dann manuell.

Abbildung 3.5 zeigt die Label Studio-Oberfläche. Alle Briefe aus der Cicero-Korrespondenz mit ihren Metadaten sind vorhanden. Pro Brief können dann bestimmte Bereiche im Text markiert und entweder als Nukleus oder als Satellit als EDUs identifiziert werden. Gleichzeitig legt man die Relationen zwischen den EDU fest. Im Bild sehen wir anhand des Briefs *Att.* 1,1 Beispiele für *List*, *Reason*, *Evidence* und *Elaboration*. Das Praktische an solchen Lösungen ist, dass man nicht alle 930 von Hand annotieren muss. Label Studio sowie Prodigy erlauben das sogenannte “Training” auf einem teilannotierten Korpus. So lernt ein System, automatisch die Annotationen vorzunehmen. Der gesamte Annotationsprozess kann so entscheidend beschleunigt werden, da die maschinell angefertigten Annotationen lediglich korrigiert werden müssen. Im Optimalfall trainiert man immer wieder neue Modelle mit bereits verifiziertem Material, sodass auch der Korrekturaufwand immer weiter abnimmt, das die automatischen Modelle eine Aufgabe besser zu lösen pflegen, wenn sie auf mehr Material trainiert worden sind.

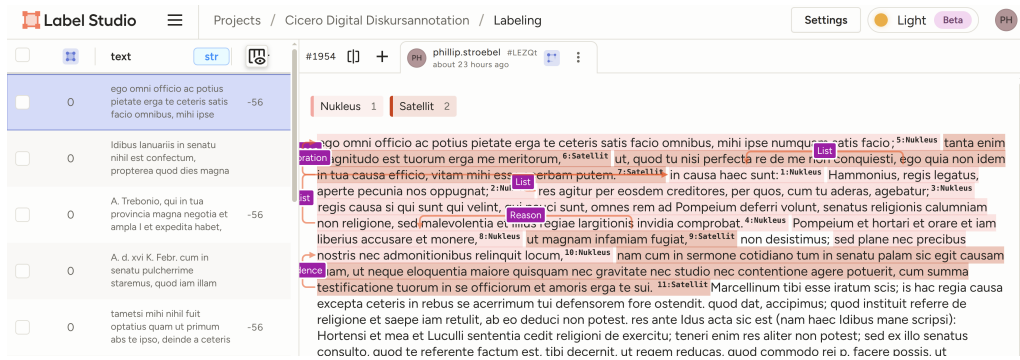
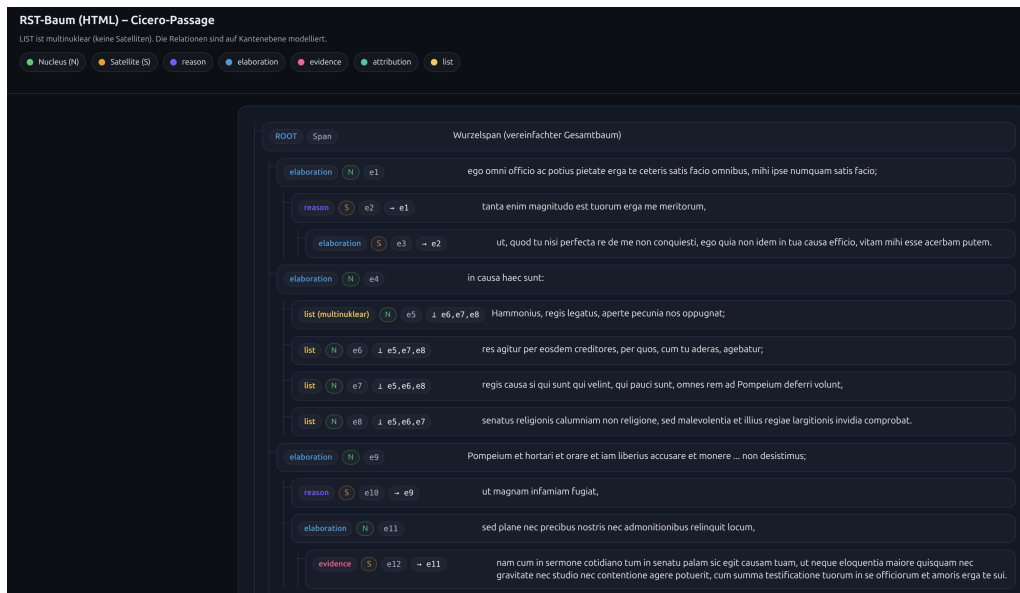


Abbildung 3.5 Screenshot aus der Label Studio-Software.

Abbildung 3.6 Beispielannotation von ChatGPT 5.2 am Brief *Att.* 1,1. Output von ChatGPT als HTML.

Es besteht natürlich auch die Möglichkeit, die Annotationsaufgabe an ein grosses Sprachmodell wie ChatGPT oder Gemini auszulagern. Aber auch hier gilt es, die automatisch erstellten Annotationen zu überprüfen. Die Abbildung 3.6 zeigt ein Resultat, das mittels des Prompts⁷

Gib mir für diesen Text die Annotation mit Nuklei und Satelliten sowie den Relationen reason, elaboration, evidence, attribution und list als Baumstruktur in einem HTML-File.

[BRIEF]

mit ChatGPT 5.2 erreicht worden ist. Der Prompt ist sehr basal und sollte zumindest die RST erwähnen. ChatGPT 5.2 merkte jedoch von selbst, dass wir auf die RST hinauswollten. Tools wie Prodigy verfügen im Übrigen bereits über die Integration von grossen Sprachmodellen, was auch hier den Annotationsprozess wesentlich beschleunigt.

Abschliessend bleibt zu sagen, dass die Diskursanalyse in Ciceros Briefwechsel enormes Potenzial bietet. Gelänge es, die gesamte Korrespondenz gemäss dem

⁷Ausgeführt am 2. März 2026.

RST-Prinzip zu annotieren, könnte man beispielsweise Aussagen darüber treffen, wie oft Cicero sich rechtfertigt, wie er es tut und bei wem am häufigsten. Solche Analysen lieferten uns Einblicke in Diskurse, die bislang möglicherweise noch gar nicht erfasst sind.

3.5 Pragmatikannotation

Kapitel
zu Prag-
matikan-
notation
schreiben

Selektion und Lücken

Lernziele Session 4: Selektion und Lücken

Verstehen Du verstehst, dass jedes digitale Korpus auf bewussten oder impliziten Selektionsentscheidungen beruht und dass Lücken, Verzerrungen und Ungleichgewichte keine Defizite, sondern zentrale Eigenschaften historischer Daten sind.

Analysieren Du kannst im Cicero-Briefkorpus erkennen, welche Briefe, Zeiträume oder Kommunikationsbeziehungen über- oder unterrepräsentiert sind und mögliche Ursachen dieser Lücken benennen (Überlieferung, Edition, Sammlungskriterien).

Anwenden Du kannst einfache Strategien anwenden, um Selektion sichtbar zu machen (z.B. Zählungen nach Zeit, Absender/Empfänger, Textlänge) und diese Ergebnisse mit deinem bisherigen Arbeitsdatensatz vergleichen.

Bewerten Du kannst beurteilen, wie sich Selektionsentscheidungen und Datenlücken auf digitale Auswertungen auswirken und welche Schlussfolgerungen historisch legitim sind – und welche nicht.

Erzeugen Du kannst eine kurze, datenbasierte Reflexion erstellen, in der du die Struktur deines Korpus beschreibst (Was ist vorhanden? Was fehlt?) und transparent machst, welche Konsequenzen dies für weitere Analysen hat.

4.1 Der ominöse Herausgeber?

Peter White macht mehrere Aspekte deutlich, die wir bereits an anderer Stelle betont haben: so zum Beispiel, dass der Briefwechsel Ciceros in vier Büchern herausgegeben wurde und dass nicht ganz klar ist, wer die Herausgeberschaft für sich beanspruchte und wie die Briefe zur Veröffentlichung überhaupt ausgewählt

wurden (siehe Unterkapitel 1.2). Wir rekapitulieren nochmals ganz kurz: Es sind um die 930 Briefe im Zeitraum von 68 v. Chr. bis 43 v. Chr. in insgesamt 36 Büchern erhalten. Die Briefe stammen von 97 benannten Briefpartnern (White, 2010, S. 172). Bezüglich des Herausgebers ist sich die Forschung uneinig, jedoch kommen mehrere Kandidaten in Betracht. Am häufigsten wird Ciceros Sekretär Tiro genannt. Das scheint eine der plausibelsten in die Herausgabe involvierte Person zu sein, hatte er doch als Cicero sehr Nahestehender Zugang zu den allermeisten Dokumenten. Als Nächstes wird Ciceros Briefpartner Atticus genannt. Auch er muss ein valabler Kandidat sein, da er zu den Korrespondenten zählt, die am meisten Briefe von Cicero erhalten haben. Ob ihm aber die Rolle als ‘Verleger’ zugeordnet werden kann, ist unsicher. Man geht eher davon aus, dass Atticus in der Vervielfältigung von Ciceros Briefen eine entscheidende Rolle gespielt hat, indem er seine Sklaven als Kopisten beschäftigte (Giebel, n. d.). Schliesslich kommt noch **Cornelius Nepos** (100–28 v. Chr.) in Frage, ein römischer Geschichtsschreiber und Biograph. Der Kontakt zu Cicero wurde von Atticus initiiert und Nepos schien direkt von Ciceros Briefen angetan gewesen zu sein. Als Schriftsteller muss er zudem über Kontakte zu Verlegern verfügt haben, was eine Rolle in der Veröffentlichung von Ciceros Briefen nachvollziehbar machen würde.

White ist sich sicher, dass die Briefe nicht von Cicero selbst veröffentlicht wurden. Gegeben der Tatsache, dass es wohl eher 70 Bücher waren und auch nur diese einen Bruchteil der gesamten Korrespondenz darstellten, also eine gewisse Selektion stattfand, kann Cicero diese Herkulesaufgabe in der ihm zur Verfügung stehenden Zeit nicht selbst bewältigt haben. Diese Selektion ist entscheidend: Wir können davon ausgehen, dass zumindest eine Person zwischen Cicero und uns steht, die unsere Rezeption Ciceros massgeblich mitprägt. Diese Person hat aktiv ausgewählt, geordnet und womöglich in den Inhalt eingegriffen. White (2010, S. 61) formuliert das so: “We cannot afford to forget that between us and Cicero’s letters stands someone who did a great deal to determine how we read them.”

Unabhängig davon, ob wir die Cicero-Briefe klassisch lesen oder mit automatisierten Verfahren analysieren, müssen wir diese Tatsache im Blick behalten. Das Auswahlverfahren beeinflusst beispielsweise stark, wie wir den in den Briefen abgebildeten Diskurs (siehe Kapitel 3) wahrnehmen und welche Themen darin vorkommen. Wiederum bei White (2010, S. 59) lesen wir: “What the editor evidently set out to construct was an epistolary portrait of a great *princeps civitatis* who over a long period dominated discourse in his milieu even if he was unable to control events.” Diese Dominanz des Diskurses, zusammen mit dem Eingriff des Editors, muss sich in der Themenwahl niederschlagen. White (2010, S. 61) stellt fest, dass viele Themen in der Korrespondenz nicht auftauchen, darunter finanzielle Angelegenheiten, Diskussionen aus dem Senat, Gerichtsverfahren, gewisse familiäre Angelegenheiten oder Briefe mit griechischen Gelehrten, von denen wir wissen, dass es sie gegeben haben muss. Was jedoch häufig vorkommt, sind dramatische politische Ereignisse wie der Bürgerkrieg oder Caesars Ermordung. Auch lesen wir einiges von prominenten Senatoren. Zudem widmet sich ein komplettes Buch (Buch 13 aus dem Subkorpus *Ad familiares*) einer speziellen

Kategorie: den Empfehlungsschreiben.

Gemäss White (2010, S. 46–51) stellt die Kategorie der Empfehlungsschreiben 10% der gesamten Korrespondenz dar. Dabei ist es bei Ciceros Briefen oft der Fall, dass sie nicht nur ein einziges Thema behandeln:

Was den Brief von anderen Textsorten unterscheidet und ihn dem mündlichen Gespräch annähert, ist die Tatsache, dass Briefe wie auch Gespräche oft nicht nur ein Thema kennen, sondern dass sich Themen – sofern es sich bei dem Meinungsaustausch nicht um ein informelles oder gar gelenktes Gespräch über ein vorher festgelegtes Thema handelt – zwanglos und gleichsam assoziativ aneinanderreihen können.“ (Rühl, 2018, S. 63)

Der Absender eines Briefes legt also eine Reihenfolge der Themen fest und bringt diese auf Papier, woraufhin der Empfänger reagiert. Jedoch muss der Empfänger in seinem Antwortschreiben nicht notwendigerweise auf die gleiche Abfolge der Themen zurückgreifen, sondern kann Themen auslassen, neue hinzufügen, und die Themen ganz generell seinem eigenen Empfinden nach ordnen. Die Themenvielfalt ist in der Cicero-Korrespondenz ausserordentlich, wie Schröder (2004, S. 193) feststellte:

Cicero schreibt über die Ereignisse in Rom, d.h. besonders im Senat, bei Wahlen, bei Prozessen, er beschreibt die allgemeine Stimmung, er gibt seine Einschätzung von Personen und Situationen, Freundschaften und Feindschaften, legt seine politischen Absichten und Pläne dar; entsprechend bittet er, wenn er selbst nicht in Rom ist, seine Adressaten um solche Informationen. Außerdem berichtet er über die Entstehung eigener literarischer Werke, er schreibt über Bauvorhaben, zu beschaffende Statuen und Bücher, über finanzielle Angelegenheiten, über Krankheiten und andere Vorkommnisse in der Familie u.v.m.

An den Inhalt, oder die in den Briefen vorkommenden Topoi, gelangen wir oft nur durch das *Close Reading*. Jedoch existieren automatische Verfahren, um an die Bedeutungsebene von Texten zu gelangen. Diese sind nun Thema im folgenden Unterkapitel.

4.2 Von TF-IDF zu Topic Modeling

Eine recht simple Methode, sich einen Überblick über möglich vorkommende Themen in einem Brief zu verschaffen, basiert zunächst auf dem *Term Frequency-Inverse Document Frequency*-Mass (TF-IDF). TF-IDF misst, wie charakteristisch ein Wort für ein Dokument—in unserem Fall ein Brief—in Relation zum Gesamtkorpus ist, und identifiziert so eigentlich die wichtigsten *Keywords* pro Brief. Die Intention dahinter ist, dass besonders häufige Wörter, die wir als Stoppwörter bezeichnen, nicht besonders aussagekräftig sind und daher einen niedrigen Score erhalten. Das gilt beispielsweise für die lateinische Konjunktion *et* (DE *und*). Die

Berechnung ist einfach. Die *Term Frequency* (TF) ist die relative Häufigkeit eines Wortes in einem Dokument. Kommt zum Beispiel das Wort *bellum* (DE *Krieg*) in einem Brief mit 120 Wörtern achtmal vor, beträgt die TF $\frac{8}{120} = 0.067$. Die *Inverse Document Frequency* (IDF) berechnet sich aus dem relativen Vorkommen eines Wortes im gesamten Korpus. So hat das Wort *bellum*, wenn wir es in 80 von 930 Briefen finden, eine IDF von $(\frac{930}{80}) = 2.45$. Seltene, distinktive Wörter erhalten hier ein höheres Gewicht. Vergleichen wir beispielsweise mit *et* und behaupten, es käme im jeden Brief vor, erhalten wir für die IDF $(\frac{930}{930}) = 1$. Den TF-IDF-Score erhalten wir, wenn wir die beiden einzelnen Werte miteinander multiplizieren: $TF \cdot IDF = TF \gg IDF$. Für unser obiges fiktives Beispiel ergäbe dies einen TF-IDF-Score von $0.067 \gg 2.45 = 0.16$. Wenn wir uns in Unterkapitel 4.3 echte Scores aus der Cicero-Korrespondenz anschauen, können wir diesen Wert besser einordnen.

Eine bisher noch nicht sehr vertiefte Literaturrecherche hat ergeben, dass TF-IDF im Bereich antiker lateinischer Texte eher selten eingesetzt wird. In einer Publikation werden die TF-IDF-Werte lediglich als Merkmale für einen Clustering-Algorithmus,¹ um Ähnlichkeiten und Unterschiede um das Thema ‘Liebe’ bei den Autoren Catull, Tibull, Properz, Vergil und Lukan aufzudecken (Nusch et al., 2025). Andere Forschungsarbeiten widmen sich der Formelhaftigkeit lateinischer Inschriften mittels TF-IDF-Massen und machen so kategorienspezifisches Vokabular offensichtlich (Soffiantini, 2024).

Das sogenannte *Topic Modeling* ist im Grunde genommen auch ein Keyword-Identifikator, fasst Keywords jedoch zu Topics zusammen und definiert pro Brief mehrere Topics. Beim Topic Modeling handelt es sich um ein sogenanntes *unsupervised* Verfahren, das auf Basis eines Textkorpus für jedes darin enthaltene Dokument eine Topic-Verteilung sowie pro Topic die aussagekräftigsten Topics berechnet. Die Topics sind dabei global zu betrachten und deren Anzahl im Vorhinein festzulegen. Das heisst, dass, wenn wir 20 Themen in einem Korpus vermuten, der Topic-Modeling-Algorithmus diese 20 Themen durch semantisch ähnliche Wörter definieren wird.

Betrachten wir das Topic Modeling ein wenig genauer. Es handelt sich um ein probabilistisches Modell. Man könnte dessen generative Story wie folgt formulieren: Wir haben zwei Würfelsets. Eines besteht aus ‘Topic-Würfeln,’ die in einem Sack vor uns liegen. Alle Topic-Würfel–genau so viele, wie wir Dokumente in unserem Korpus haben–besitzen exakt k Seiten, wobei k für die Anzahl der zu inferierenden Topics steht. Das zweite Set besteht aus den ‘Wort-Würfeln,’ wobei wir für jedes Topic einen Würfel verzeichnen (wir haben also k Wort-Würfel). Jeder Wort-Würfel hat n Seiten, wobei n der Anzahl der Wörter des Vokabulars entspricht. Wichtig ist, dass wir noch anmerken, dass die Seiten beider Würfelarten nicht gleich gross sind. So kann das Topic ‘Krieg’ auf einem Würfel 40% der Fläche einnehmen, während es auf einem anderen bloss 5% der Fläche besetzt. Genauso auf den Wort-Würfeln: Das Wort *bellum* ist auf dem Topic-Würfel ‘Liebe’ nicht gleich wahrscheinlich wie auf dem Topic-Würfel ‘Krieg.’ Nun wollen wir ein 100-wörtiges Dokument generieren. Hierbei gilt es, zunächst die Topic-Verteilung festzulegen. Dazu ziehen wir zuerst einen

¹Wir wenden uns dem Clustering noch detaillierter im Kapitel ?? zu.

Würfel aus dem Sack mit den Topic-Würfeln. Nun generieren wir die Wörter des Dokuments, und zwar in einem iterativen Prozess. Wir nehmen zuerst den Topic-Würfel und bestimmen das Topic durch einen Wurf (zum Beispiel ‘Krieg’). Somit nehmen wir zuerst den ‘Krieg’-Würfel und würfeln ein Wort (zum Beispiel *bellum*). Diesen Vorgang wiederholen wir nun, bis unser Dokument 100 Wörter lang ist. Abbildung 4.1 stellt diesen Prozess schematisch dar.

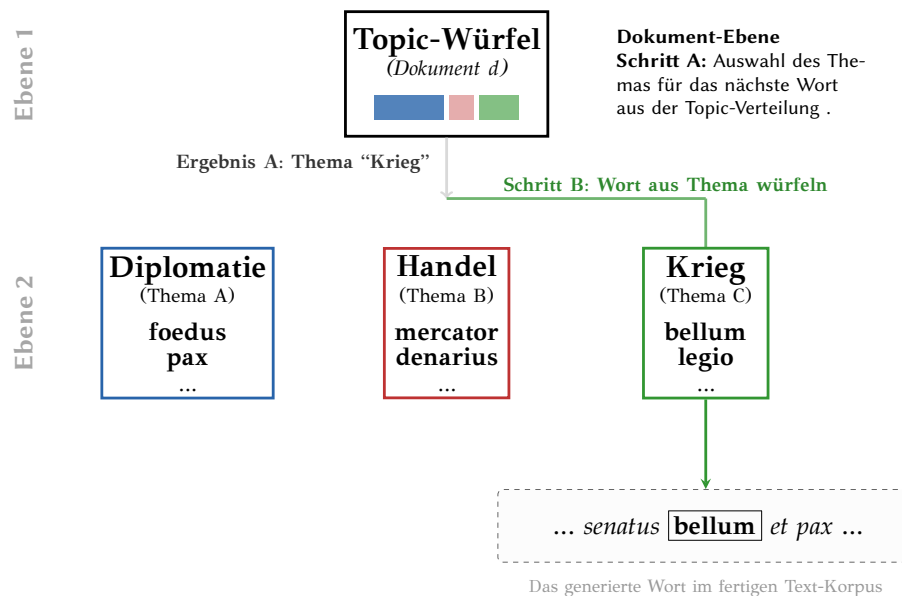


Abbildung 4.1 Der zweistufige generative Prozess von LDA: Von der dokumentweiten Themenwahl (Ebene 1) hin zur spezifischen Wortziehung aus dem Themen-Pool (Ebene 2).

Für das Topic Modeling müssen wir das nun umgekehrt machen. Ein Algorithmus, der dazu imstande ist, heisst Latent Dirichlet Allocation (Blei et al., 2003). Sie versteht die Entstehung eines historischen Dokuments als ein stochastisches Experiment. Das Dokument liegt fertig vor uns, und wir wollen wissen, welche Würfel der (hypothetische) Autor verwendet hat. Da wir den Prozess nicht beobachten konnten, nutzen wir ein Näherungsverfahren, das *Gibbs Sampling*:

Initialisierung: Wir gehen durch den Text und weisen jedem Wort völlig zufällig ein Thema zu. Dieser Zustand ist pures Rauschen.

Iterative Neuzuweisung: Wir nehmen ein einzelnes Wort aus dem Text heraus und prüfen: “Zu welchem Thema passt dieses Wort am besten?” Dabei berücksichtigt der Computer zwei Faktoren:

1. Wie viele Wörter desselben Themas befinden sich bereits in diesem Dokument?
2. Wie oft wurde dieses spezifische Wort im restlichen Korpus diesem Thema zugeordnet?

Stabilisierung: Dieser Vorgang wird für jedes Wort im Korpus tausende Male wiederholt. Wie bei einem sozialen System, das sich einpendelt, sortieren sich die Wörter allmählich in semantisch sinnvolle Gruppen. Am Ende

“kristallisieren” sich die Themen heraus, ohne dass wir dem Computer vorher sagen mussten, was ein Thema inhaltlich bedeutet.

Hier dient Abbildung 4.2 als Illustration.

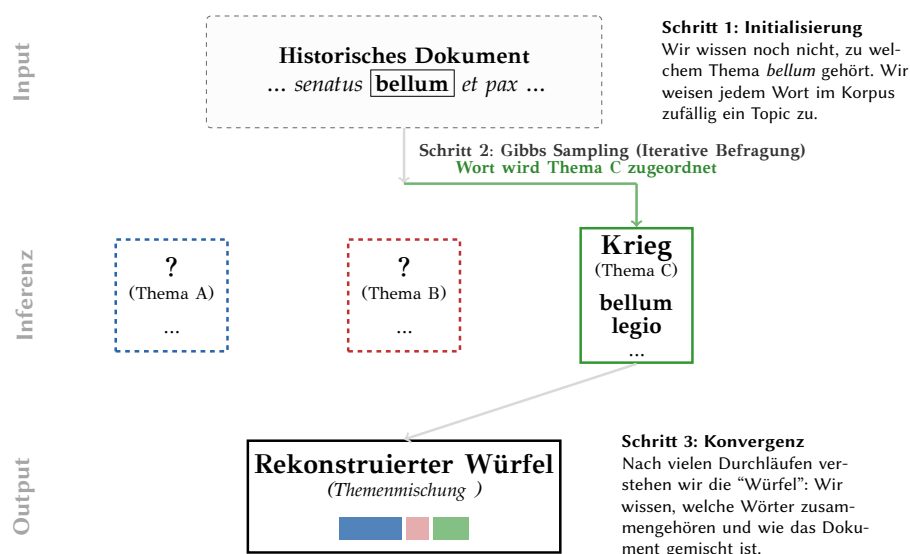


Abbildung 4.2 Der Inferenz-Prozess (Reverse Engineering): Aus den beobachteten Wörtern des Dokuments werden die latenten Themen-Strukturen und die spezifische Topic-Verteilung rekonstruiert.

Die dadurch entstandenen Topics sind im Nachhinein natürlich zu prüfen. Sie bestehen sozusagen aus ‘Wortwolken,’ denen man zuerst einen Übergriff als Topic zuordnen muss. Die allgemeine Interpretation von solchen Topics wurde in der Vergangenheit auch schon als “Kaffesatzlesen” bezeichnet (Chang et al., 2009). Es braucht also noch einen interpretativen Akt, bevor man Topic Models aktiv nutzen kann. Wenn erfolgreich “trainiert,” lassen sich die Topics aber beispielsweise als Filter nutzen, wie vom *impresso*-Projekt demonstriert.²

4.3 TF-IDF

Nachdem wir im vorherigen Unterkapitel das Grundprinzip von TF-IDF theoretisch eingeführt haben, wenden wir das Verfahren nun auf das vorverarbeitete Korpus der Cicero-Briefe an. Ziel ist es nicht, Themen im strengen Sinn automatisch zu erkennen, sondern zunächst jene Wörter sichtbar zu machen, die für einzelne Briefe oder für Gruppen von Briefen besonders charakteristisch sind.

TF-IDF eignet sich vor allem für eine erste, explorative Annäherung an ein Korpus. Das Verfahren hebt Wörter hervor, die in einem bestimmten Dokument vergleichsweise häufig vorkommen, im Gesamtkorpus jedoch nicht allzu verbreitet sind. Damit erhalten wir keine “Interpretation” des Textes, wohl aber einen Hinweis darauf, welches Vokabular einen Brief semantisch prägt.

Das dazugehörige Jupyter Notebook befindet sich in der JupyterLite-Umgebung.

²Siehe <https://dev.impresso-project.ch/app>. Das *impresso*-Projekt widmet sich der semantischen Indexierung von mehreren Zeitungsarchiven, um diese für Forschende zugänglich zu machen.

4.3.1 Ziel dieses Abschnitts

Wir verfolgen drei Teilziele:

1. Wir berechnen TF-IDF-Werte für die Briefe des Korpus.
2. Wir lassen uns die charakteristischsten Wörter einzelner Briefe anzeigen.
3. Wir nutzen die TF-IDF-Repräsentation, um Ähnlichkeiten zwischen Briefen sichtbar zu machen.

Dabei ist wichtig: TF-IDF arbeitet auf der Ebene von Wortformen oder normalisierten Tokens. Das Verfahren “versteht” keinen Text im menschlichen Sinn, sondern gewichtet Wörter mathematisch. Die Interpretation bleibt daher **ein historisch-philologischer Schritt**.

4.3.2 Daten laden

Wir arbeiten mit einem bereits vorbereiteten DataFrame, in dem pro Brief eine Zeile enthalten ist. Zentral ist dabei eine bereinigte Textspalte, die wir für die Vektorisierung verwenden.

Code-Beispiel 4.1 Bereinigtes Korpus laden.

```
from pathlib import Path
import pandas as pd

csv_path = Path("outputs") / "cicero_letters_clean.csv"
df = pd.read_csv(csv_path)

df[["doc_id", "corpus", "clean_text"]].head()
```

Die Spalte `clean_text` enthält den für TF-IDF vorbereiteten Text. Je nach Vorverarbeitung können darin Interpunktionszeichen, sehr häufige Funktionswörter oder andere Störfaktoren bereits entfernt worden sein. Solche Entscheidungen sind nicht neutral, sondern beeinflussen direkt, welche Wörter später als ‘charakteristisch’ erscheinen.

4.3.3 TF-IDF-Vektorisierung

Für die Berechnung verwenden wir den `TfidfVectorizer` aus `scikit-learn`. Dieser transformiert die Texte in eine Matrix, in der jede Zeile einem Brief und jede Spalte einem Wort des Vokabulars entspricht.

Vektori-
sierung
erklären.

Code-Beispiel 4.2 TF-IDF-Matrix berechnen.

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(
    lowercase=False,
    max_df=0.8,
    min_df=2
```

```
)

X = vectorizer.fit_transform(df["clean_text"])
feature_names = vectorizer.get_feature_names_out()

X.shape
```

Die Matrix X ist typischerweise sehr gross, aber auch sehr dünn besetzt. Das heisst, dass viele Einträge eine 0 aufweisen. Das liegt daran, dass jeder einzelne Brief nur einen kleinen Teil des gesamten Vokabulars enthält. Man spricht deshalb im Englischen von einer *sparse matrix*.

Die Parameter `max_df` und `min_df` helfen uns dabei, extrem häufige oder extrem seltene Wörter auszublenden. So reduzieren wir Rauschen und konzentrieren uns auf Wörter, die für Vergleiche sinnvoller sind.

4.3.4 Top-Wörter eines einzelnen Briefs

Ein besonders naheliegender erster Schritt besteht darin, sich für einen Brief die höchsten TF-IDF-Werte anzeigen zu lassen. So erhalten wir eine Liste von Wörtern, die gerade diesen Brief besonders stark prägen.

Code-Beispiel 4.3 Wichtigste TF-IDF-Wörter eines Briefs ausgeben.

```
import numpy as np

def top_tfidf_words(doc_index, n=10):
    row = X[doc_index].toarray().flatten()
    top_idx = row.argsort()[::-1][:n]
    return [(feature_names[i], row[i]) for i in top_idx if row[i] > 0]

doc_index = 0
top_tfidf_words(doc_index, n=10)
```

Das Resultat ist eine Rangliste von Wörtern mitsamt ihren Gewichten. Historisch interessant ist nun die Frage, ob diese Wörter tatsächlich einen Themenkern treffen oder ob sie bloss zufällige, situationsgebundene Besonderheiten des jeweiligen Briefs abbilden.

[6]:	term	score
0	pario	0.223502
1	soror	0.168533
2	signum	0.166513
3	comitia	0.159314
4	locus	0.143989
5	confido	0.135665
6	proficiscor facio	0.130256
7	communis amicus	0.130256
8	sponsor appello	0.130256
9	causa magnus	0.130256

Abbildung 4.3 Charakteristische TF-IDF-Wörter eines Beispielbriefs.

4.3.5 Die wichtigsten TF-IDF-Wörter im gesamten Korpus

Nebst der Einzelperspektive können wir auch untersuchen, welche Wörter korpusweit hohe TF-IDF-Werte akkumulieren. Dazu summieren wir die Werte spaltenweise auf.

Code-Beispiel 4.4 Korpusweit wichtige TF-IDF-Wörter berechnen.

```
tfidf_sum = np.asarray(X.sum(axis=0)).flatten()
top_idx = tfidf_sum.argsort()[::-1][:20]

top_words_corpus = [
    (feature_names[i], tfidf_sum[i])
    for i in top_idx
]

top_words_corpus
```

Diese Liste zeigt nicht die häufigsten Wörter des Korpus, sondern jene, die über viele Dokumente hinweg besonders aussagekräftig erscheinen. In der Praxis stehen hier oft politische Akteure, Ortsnamen oder thematisch markierte Substantive.

	term	score
0	uenio	28.274562
1	puto	26.996753
2	dies	24.859090
3	ago	24.768584
4	dico	24.645316
5	scio	24.627707
6	do	23.051622
7	causa	22.156872
8	magnus	21.985530
9	caesar	21.645382
10	homo	21.343235
11	mitto	20.119423
12	tempus	19.860796
13	consilium	18.904614
14	senatus	18.506030
15	animus	18.369162
16	ualeo	18.238813
17	modus	17.531360
18	exspecto	16.930508
19	brutus	16.038638

Abbildung 4.4 Korpusweit auffällige Wörter nach aufsummiertem TF-IDF.

4.3.6 Ähnliche Briefe mit Cosine Similarity

Ein grosser Vorteil der TF-IDF-Repräsentation besteht darin, dass sich Briefe nun mathematisch vergleichen lassen. Zwei Briefe gelten dann als ähnlich, wenn sie ähnliche gewichtete Wortprofile aufweisen. Ein gängiges Mass dafür ist die *Cosine Similarity*.

Code-Beispiel 4.5 Ähnlichkeiten zwischen Briefen berechnen.

```
from sklearn.metrics.pairwise import cosine_similarity

sim_matrix = cosine_similarity(X)

target_doc = df.loc[0, "doc_id"]
target_idx = df.index[df["doc_id"] == target_doc][0]

scores = sim_matrix[target_idx]
top_idx = scores.argsort()[::-1][1:11]

similar_docs = df.loc[top_idx, ["doc_id", "corpus"]].copy()
similar_docs["similarity"] = scores[top_idx]
similar_docs
```

Hier wird für einen ausgewählten Brief eine Liste der ähnlichsten Briefe erstellt. Solche Resultate können auf inhaltliche Nähe hinweisen, etwa auf gemeinsame politische Situationen, wiederkehrende Empfehlungsschreiben oder ähnliche Adressatenkonstellationen. Gleichzeitig ist Vorsicht geboten: Eine hohe Ähnlichkeit kann ebenso aus formelhaften Wendungen oder editorischen Gemeinsamkeiten resultieren.

4.3.7 Visualisierung eines Similarity-Ausschnitts

Um die Struktur des Korpus besser zu erfassen, kann ein Ausschnitt der Ähnlichkeitsmatrix visualisiert werden.

Code-Beispiel 4.6 Heatmap für einen Ausschnitt der Briefähnlichkeiten.

```
import matplotlib.pyplot as plt
import seaborn as sns

max_docs = min(30, len(df))
subset = sim_matrix[:max_docs, :max_docs]

plt.figure(figsize=(10, 8))
sns.heatmap(subset, cmap="viridis")
plt.title("Cosine Similarity zwischen den ersten Briefen")
plt.xlabel("Briefindex")
plt.ylabel("Briefindex")
plt.show()
```

Solche Heatmaps helfen dabei, Cluster ähnlicher Briefe zu erkennen. Wo sich hellere Blöcke oder dichtere Felder bilden, könnte eine thematische oder

stilistische Gruppierung vorliegen. Diese muss anschliessend aber durch Close Reading überprüft werden.

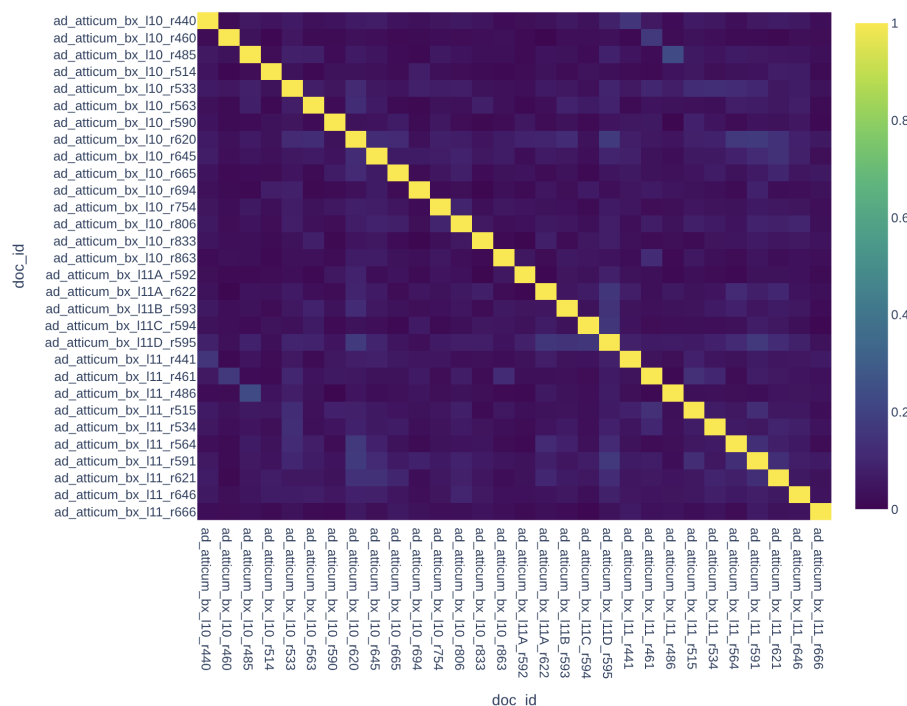


Abbildung 4.5 Ähnlichkeitsstruktur eines Korpusausschnitts auf Basis von TF-IDF.

4.3.8 Ein kleines Netzwerk ähnlicher Briefe

Aus den Ähnlichkeiten lässt sich schliesslich auch ein Netzwerk erzeugen. Verbunden werden dabei nur jene Briefe, deren Ähnlichkeit einen bestimmten Schwellenwert überschreitet.

Code-Beispiel 4.7 Netzwerk ähnlicher Briefe erzeugen.

```
import networkx as nx

threshold = 0.20
G = nx.Graph()

for i, row in df.iterrows():
    G.add_node(row["doc_id"], corpus=row["corpus"])

for i in range(len(df)):
    for j in range(i + 1, len(df)):
        if sim_matrix[i, j] >= threshold:
            G.add_edge(df.loc[i, "doc_id"], df.loc[j, "doc_id"],
                      weight=sim_matrix[i, j])

len(G.nodes()), len(G.edges())
```

Ein solches Netzwerk bildet keine historische Kommunikationsbeziehung ab, sondern eine *textuelle Ähnlichkeitsrelation*. Es zeigt also nicht, wer mit wem

korrespondiert hat, sondern welche Briefe sich im verwendeten Vokabular oder im thematischen Profil ähneln.

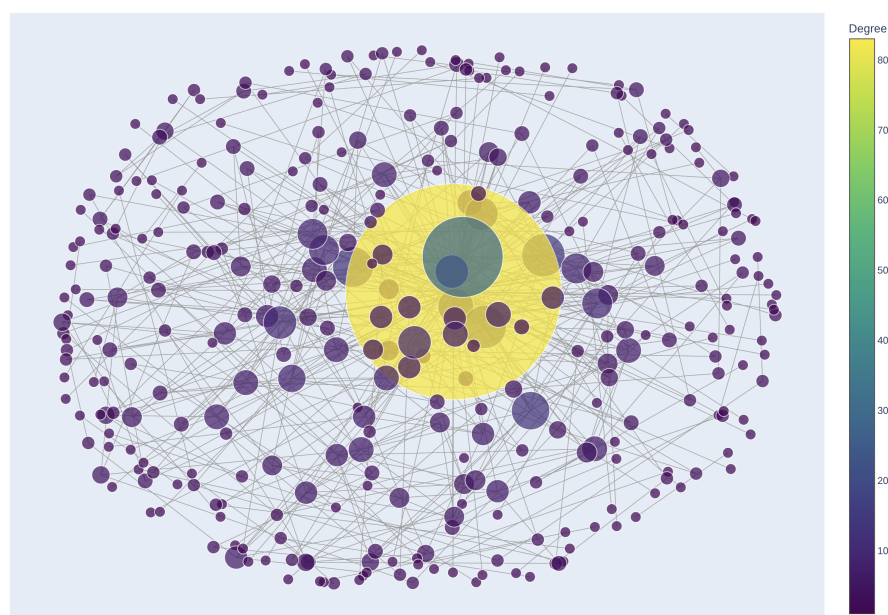


Abbildung 4.6 Textuelles Ähnlichkeitsnetzwerk der Briefe.

4.3.9 Fazit

TF-IDF ist ein nützliches Verfahren für den ersten Zugriff auf ein Korpus. Es hilft uns,

- charakteristische Wörter einzelner Briefe sichtbar zu machen,
- das Vokabular mehrerer Briefe miteinander zu vergleichen,
- und Ähnlichkeitsstrukturen im Korpus explorativ zu untersuchen.

Gerade für historische Korpora ist TF-IDF deshalb wertvoll, weil das Verfahren einfach, transparent und reproduzierbar ist. Zugleich ersetzt es keine Interpretation. Es zeigt uns auffällige Wörter, aber nicht automatisch, wie diese historisch zu deuten sind. Für eine stärker themenorientierte Modellierung wenden wir uns deshalb nun dem Topic Modeling zu.

4.4 Topic Modeling

Während TF-IDF in erster Linie charakteristische Wörter hervorhebt, versucht Topic Modeling, aus einem Korpus wiederkehrende thematische Muster zu erschliessen. Dabei werden Wörter, die häufig gemeinsam auftreten, zu *Topics* gebündelt. Ein Topic ist somit keine fertige historische Kategorie, sondern zunächst eine statistische Wortgruppe, die wir erst im Nachhinein interpretieren müssen.

Im Folgenden arbeiten wir zunächst mit einem klassischen LDA-Modell, das mit *gensim* (Řehůřek & Sojka, 2010) trainiert wird. Anschliessend betrachten wir

mit BERTopic (Grootendorst, 2022) einen neueren Ansatz, der stärker semantisch arbeitet.

4.4.1 Vorbereitung des Korpus

Für das Topic Modeling müssen die Briefe in tokenisierter Form vorliegen. Zusätzlich erstellen wir ein Wörterbuch sowie eine Bag-of-Words-Repräsentation.

BoW er-
klären

Code-Beispiel 4.8 Korpus für gensim vorbereiten.

```
from gensim import corpora

texts = [doc.split() for doc in df["clean_text"]]

dictionary = corpora.Dictionary(texts)
dictionary.filter_extremes(no_below=2, no_above=0.8)

corpus = [dictionary.doc2bow(text) for text in texts]

len(dictionary), len(corpus)
```

Hier zeigt sich ein wichtiger Unterschied zu TF-IDF: Für das LDA-Modell arbeiten wir nicht mit gewichteten Vektoren auf der Basis von TF-IDF, sondern mit einer reduzierten Bag-of-Words-Darstellung. Entscheidend ist dabei, welche Wörter überhaupt im Wörterbuch verbleiben. Auch diese Auswahl beeinflusst später die Topics.

4.4.2 LDA-Modell mit gensim

Wir trainieren nun ein Topic-Modell mit einer festgelegten Anzahl von Topics. Gerade diese Zahl ist ein methodischer Eingriff: Sie ergibt sich nicht automatisch aus dem Korpus, sondern muss explorativ gewählt und geprüft werden.

Code-Beispiel 4.9 LDA-Modell trainieren.

```
from gensim.models import LdaModel

NUM_TOPICS = 20

lda = LdaModel(
    corpus=corpus,
    id2word=dictionary,
    num_topics=NUM_TOPICS,
    random_state=42,
    passes=10,
    iterations=100
)
```

Das trainierte Modell enthält nun für jedes Topic eine Liste von besonders wahrscheinlichen Wörtern. Diese Wörter bilden die Grundlage für unsere Interpretation.

4.4.3 Top-Wörter pro Topic anzeigen

Ein erster Blick auf das Modell besteht darin, sich für jedes Topic die wichtigsten Wörter ausgeben zu lassen.

Code-Beispiel 4.10 Top-Wörter pro Topic ausgeben.

```
for i, topic in lda.print_topics(num_topics=NUM_TOPICS, num_words=10):
    print(f"Topic {i}: {topic}")
```

Das Resultat sind Wortlisten, die man als eine Art Wortwolken in Textform lesen kann. Ein Topic könnte beispielsweise durch Wörter aus dem Feld *politische Krise*, *Freundschaft*, *Empfehlung* oder *Reise* geprägt sein. Diese Bezeichnungen entstehen jedoch nicht im Modell selbst, sondern werden von uns interpretativ vergeben. So könnte beispielsweise ein Label für das Topic 6 in Abbildung 4.7 “politische Angelegenheiten” heissen, da die wichtigsten Wörter darin *prouincia* (DE “Provinz”) und *senatus* (DE “Senat”) sind. Zugleich fällt auf, dass in vielen Topics die Wörter *do* (DE “geben”), *uenio* (DE “kommen”) oder *puto* (DE “glauben”) dominieren. Diese sind können wir als korpuspezifische Stoppwörter bezeichnen und diese sollten vor dem Topic Modeling herausgefiltert werden, um aussagekräftigere Topics zu erhalten.

4.4.4 Kohärenz des Modells prüfen

Nicht jedes Topic-Modell ist gleich gut interpretierbar. Ein gebräuchliches Hilfsmittel ist die sogenannte Topic-Kohärenz. Sie misst vereinfacht gesagt, wie gut die Top-Wörter eines Topics zusammenpassen.

Code-Beispiel 4.11 Kohärenz des LDA-Modells berechnen.

```
from gensim.models import CoherenceModel

coherence_model = CoherenceModel(
    model=lda,
    texts=texts,
    dictionary=dictionary,
    coherence="c_v"
)

coherence_score = coherence_model.get_coherence()
coherence_score
```

Der Kohärenzwert hilft uns beim Vergleich mehrerer Modelle, etwa mit 10, 15, 20 oder 25 Topics. Er ersetzt jedoch nicht die inhaltliche Prüfung. Ein mathematisch “gutes” Modell kann historisch dennoch unplausible oder zu grobe Topics produzieren.

4.4.5 Dominantes Topic pro Brief

Ein LDA-Modell weist jedem Brief nicht bloss ein einziges Thema zu, sondern eine Verteilung über mehrere Topics. Für eine vereinfachte Darstellung können

```

Topic 0: 0.011*“consilium”+ 0.010*“animus”+ 0.009*“puto”+
0.008*“tempus”+ 0.007*“debeo”+ 0.007*“bellum”+ 0.007*“ma-
gnus”+ 0.007*“ago”+ 0.007*“caesar”+ 0.007*“publicus”
Topic 1: 0.012*“liber”+ 0.011*“fio”+ 0.010*“ago”+ 0.010*“puto”+
0.009*“nolo”+ 0.008*“uenio”+ 0.007*“loquor”+ 0.007*“bellum”+
0.007*“do”+ 0.007*“tempus”
Topic 2: 0.016*“dies”+ 0.014*“uenio”+ 0.013*“scio”+ 0.012*“cae-
sar”+ 0.012*“puto”+ 0.011*“ago”+ 0.011*“pompeius”+ 0.011*“do”+
0.009*“dico”+ 0.008*“homo”
Topic 3: 0.023*“uenio”+ 0.010*“do”+ 0.010*“mitto”+ 0.010*“epistu-
la”+ 0.010*“exspecto”+ 0.009*“puto”+ 0.009*“dico”+ 0.009*“ago”+
0.008*“modus”+ 0.007*“antonius”
Topic 4: 0.013*“puto”+ 0.010*“scio”+ 0.010*“uenio”+ 0.009*“tempus”+
0.009*“ualeo”+ 0.008*“audio”+ 0.007*“homo”+ 0.007*“magnus”+
0.006*“amo”+ 0.006*“animus”
Topic 5: 0.016*“caesar”+ 0.013*“ago”+ 0.013*“scio”+ 0.011*“uenio”+
0.011*“do”+ 0.009*“consul”+ 0.008*“tempus”+ 0.008*“dico”+
0.008*“laus”+ 0.008*“dies”
Topic 6: 0.020*“prouincia”+ 0.015*“senatus”+ 0.011*“mitto”+ 0.010*“gai-
us”+ 0.009*“dico”+ 0.008*“magnus”+ 0.008*“puto”+ 0.007*“dies”+
0.007*“uenio”+ 0.007*“causa”
Topic 7: 0.016*“do”+ 0.014*“dies”+ 0.011*“dico”+ 0.010*“homo”+
0.009*“accipio”+ 0.008*“consul”+ 0.008*“pompeius”+ 0.008*“uenio”+
0.008*“idus”+ 0.007*“senatus”
Topic 8: 0.014*“puto”+ 0.012*“ago”+ 0.011*“homo”+ 0.010*“uenio”+
0.009*“scio”+ 0.008*“consilium”+ 0.008*“do”+ 0.008*“dico”+ 0.007*“in-
tellego”+ 0.007*“causa”
Topic 9: 0.015*“scio”+ 0.015*“legio”+ 0.013*“antonius”+ 0.012*“uenio”+
0.010*“do”+ 0.010*“dies”+ 0.010*“puto”+ 0.009*“castra”+ 0.009*“ua-
leo”+ 0.008*“prouincia”

```

Abbildung 4.7 Beispielhafte LDA-Topics (gensim-Output der ersten 10 Topics) aus dem Cicero-Korpus.

wir jedoch das dominante Topic pro Brief bestimmen.

Code-Beispiel 4.12 Dominantes Topic pro Brief bestimmen.

```
def dominant_topic_for_bow(bow):
    topics = lda.get_document_topics(bow)
    if not topics:
        return None
    return max(topics, key=lambda x: x[1])[0]

df["dominant_topic"] = [dominant_topic_for_bow(bow) for bow in corpus]

df[["doc_id", "corpus", "dominant_topic"]].head()
```

Diese Reduktion ist praktisch für Übersichten, aber sie vereinfacht die Textrealität stark. Gerade Briefe können, wie wir gesehen haben, mehrere Themen zugleich verhandeln. Die Rede vom “dominanten Topic” ist daher eine analytische Setzung.

4.4.6 Verteilung der Topics visualisieren

Sobald jedem Brief ein dominantes Topic zugewiesen ist, können wir untersuchen, welche Topics im Korpus besonders häufig auftreten.

Code-Beispiel 4.13 Verteilung dominanter Topics visualisieren.

```
topic_counts = df["dominant_topic"].value_counts().sort_index()

plt.figure(figsize=(10, 5))
topic_counts.plot(kind="bar")
plt.title("Dominantes Topic pro Brief")
plt.xlabel("Topic")
plt.ylabel("Anzahl Briefe")
plt.show()
```

Eine solche Verteilung kann auf thematische Schwerpunkte des überlieferten Korpus hinweisen. Gerade im Kontext von White ist dies besonders spannend: Wenn bestimmte Themen deutlich dominieren und andere fast gar nicht vorkommen, dann kann dies ein Echo editorischer Selektion sein und nicht bloss ein Abbild von Ciceros tatsächlicher Kommunikationspraxis. Der Reader betont bereits in den Unterkapiteln 4.1 und 4.2, dass Auswahl, Ordnung und Überlieferung den Blick auf den Briefwechsel entscheidend prägen.

4.4.7 Top-Wörter pro Topic als Balkendiagramm

Nebst der textuellen Ausgabe lassen sich die wichtigsten Wörter pro Topic auch grafisch darstellen.

Code-Beispiel 4.14 Top-Wörter pro Topic visualisieren.

```
top_n = 10
```

	doc_id	dominant_topic	topic_prob
0	ad_atticum_bx_l10_r440	14	0.642762
1	ad_atticum_bx_l10_r460	10	0.753302
2	ad_atticum_bx_l10_r485	10	0.543301
3	ad_atticum_bx_l10_r514	14	0.592924
4	ad_atticum_bx_l10_r533	4	0.789066

Abbildung 4.8 Häufigkeit der dominanten Topics pro Brief.

```
for topic_id in range(NUM_TOPICS):
    words_probs = lda.show_topic(topic_id, topn=top_n)
    words = [w for w, _ in words_probs]
    probs = [p for _, p in words_probs]

    plt.figure(figsize=(8, 4))
    plt.barh(words[::-1], probs[::-1])
    plt.title(f"Topic {topic_id}")
    plt.xlabel("Wahrscheinlichkeit")
    plt.show()
```

Solche Diagramme erleichtern die qualitative Benennung der Topics. Oft erkennt man bereits auf einen Blick, ob eher politische, personale, geographische oder epistolografische Wortfelder im Vordergrund stehen.

4.4.8 Dokument-Topic-Matrix

Schliesslich können wir für jeden Brief die gesamte Topic-Verteilung speichern. So entsteht eine Matrix, in der jeder Brief als Mischung mehrerer Topics sichtbar wird.

Code-Beispiel 4.15 Dokument-Topic-Matrix erzeugen.

```
doc_topic_matrix = []

for bow in corpus:
    dist = lda.get_document_topics(bow, minimum_probability=0.0)
    row = {f"topic_{topic_id}": prob for topic_id, prob in dist}
    doc_topic_matrix.append(row)

doc_topic_df = pd.DataFrame(doc_topic_matrix)
doc_topic_df.head()
```

Diese Matrix kann später mit Metadaten verknüpft werden. So liesse sich etwa fragen, ob gewisse Topics in bestimmten Teilkorpora, Jahren oder Adressantenkonstellationen gehäuft auftreten.

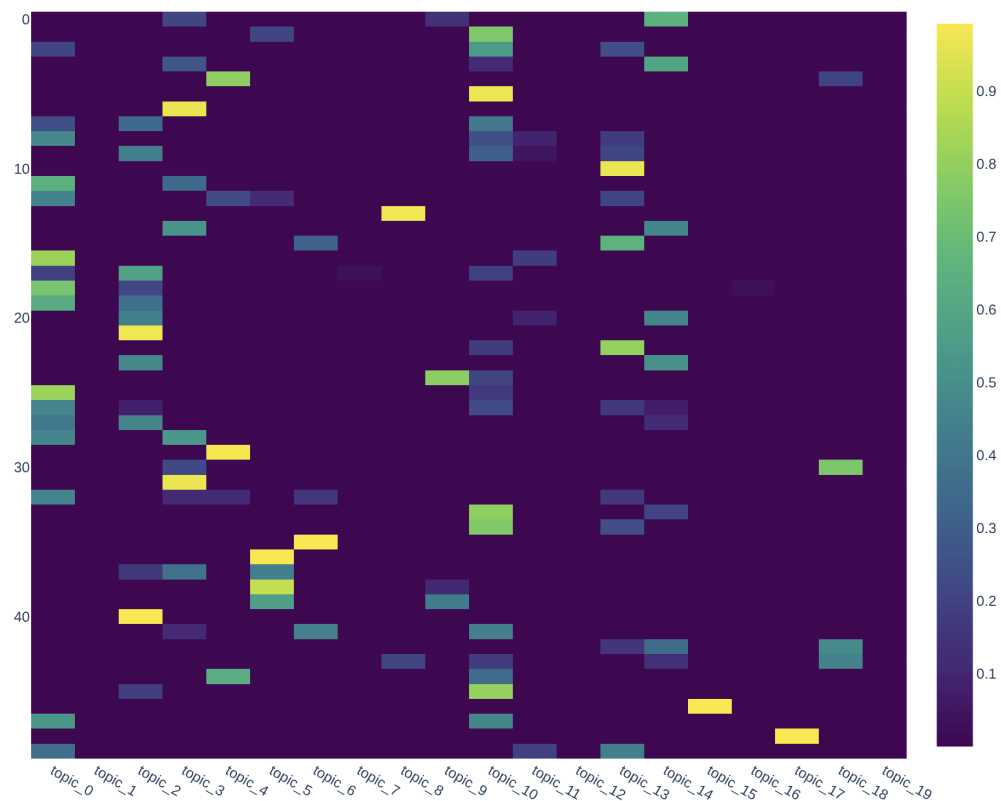


Abbildung 4.9 Themenmischungen mehrerer Briefe.

4.4.9 Interaktive Visualisierung mit pyLDAvis

Für explorative Zwecke ist eine interaktive Darstellung des Topic-Modells hilfreich. pyLDAvis zeigt die Topics im Raum sowie die wichtigsten Wörter pro Topic.

Code-Beispiel 4.16 Interaktive pyLDAvis-Visualisierung vorbereiten.

```
import pyLDAvis
import pyLDAvis.gensim_models as gensimvis

vis = gensimvis.prepare(lda, corpus, dictionary)
pyLDAvis.display(vis)
```

Die Visualisierung erlaubt es, Topics hinsichtlich ihrer Nähe und Trennschärfe zu vergleichen. Wenn zwei Topics stark überlappen, ist das oft ein Hinweis darauf, dass die gewählte Anzahl Topics noch nicht optimal ist.

4.4.10 BERTopic

Nebst klassischen Topic-Modellen wie LDA gibt es neuere Verfahren, die stärker semantisch arbeiten. Eines davon ist BERTopic. Während LDA auf Wortverteilungen basiert, nutzt BERTopic in der Regel dichte Vektorrepräsentationen (sogenannte Embeddings), um semantisch ähnliche Dokumente zu gruppieren. Anschliessend werden diese Cluster mit charakteristischen Wörtern beschrieben. Hier kann beispielsweise auch wieder TF-IDF ins Spiel kommen.

Der Vorteil eines solchen Ansatzes liegt darin, dass nicht bloss identische oder ähnliche Wortformen erfasst werden, sondern stärker auch semantische Nähe. Gerade für Briefe, in denen ein Thema mit variierendem Vokabular angesprochen werden kann, ist das potenziell interessant. Der Nachteil besteht darin, dass das Verfahren methodisch weniger transparent ist als klassische Bag-of-Words-Modelle und in der Praxis stärker von den verwendeten Embeddings abhängt.

BERTopic integrieren

LDA und BERTopic im Vergleich

LDA und BERTopic verfolgen dasselbe Ziel, nämlich thematische Strukturen im Korpus sichtbar zu machen, tun dies aber auf unterschiedliche Weise:

- LDA ist stärker wortbasiert, einfacher zu erklären und methodisch sehr transparent.
- BERTopic arbeitet semantischer und kann flexibler auf unterschiedliche Ausdrucksweisen reagieren.
- Beide Verfahren produzieren jedoch keine fertigen historischen Wahrheiten, sondern interpretierbare Modellierungen.

Für den Einsatz im Unterricht ist LDA oft der bessere Ausgangspunkt, weil das Verfahren enger an den beobachtbaren Wortverteilungen bleibt. BERTopic eignet sich besonders dann als Ergänzung, wenn man demonstrieren möchte, wie moderne embeddingsbasierte Verfahren thematische Nähe modellieren.

4.4.11 Fazit

Topic Modeling ermöglicht es uns, im Cicero-Korpus wiederkehrende thematische Muster sichtbar zu machen. Gerade für ein grosses Briefkorpus ist dies attraktiv, weil wir nicht jeden Brief isoliert lesen müssen, um erste thematische Strukturen zu erkennen.

Gleichzeitig bleibt festzuhalten:

- Topics sind keine natürlichen Einheiten, sondern Modellresultate.
- Die Zahl der Topics, die Vorverarbeitung und die Wahl des Verfahrens prägen das Ergebnis stark.
- Die Interpretation der Topics bleibt ein philologischer und historischer Akt.

Damit eignet sich Topic Modeling besonders gut als Brücke zwischen Distant Reading und Close Reading: Es zeigt uns, *wo* thematische Verdichtungen liegen könnten. Was diese historisch bedeuten, muss anschliessend wieder am Text selbst geprüft werden.

Literatur

- Babeu, A., & Dilley, P. (2021). Linked Open Data for Greek and Latin Authors and Works. Verfügbar 23. Februar 2026 unter <https://dlib.nyu.edu/awdl/isaw/isaw-papers/20-9/>
- Bamman, D., Passarotti, M., Crane, G., & Raynaud, S. (2007). A Collaborative Model of Treebank Development. Verfügbar 23. Februar 2026 unter <http://hdl.handle.net/10062/4755>
- Becker, E.-M., & Blom, H. v. d. (2025). Epistolary Communication and Selffashioning: Cicero and Paul in Comparison. In E.-M. Becker, U. Egelhaaf-Gaiser & A. Fürst (Hrsg.), *Handbuch Brief: Antike* (S. 133–148). De Gruyter. Verfügbar 8. Februar 2026 unter <https://www.degruyterbrill.com/document/doi/10.1515/9783110586800-006/html>
- Becker, E.-M., Egelhaaf-Gaiser, U., & Fürst, A. (Hrsg.). (2025). *Handbuch Brief: Antike*. Walter de Gruyter GmbH & Co KG.
- Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent Dirichlet Allocation. *Journal of Machine Learning Research*, 3(Jan), 993–1022. <http://www.jmlr.org/papers/v3/blei03a.html>
- Brinton, L. J. (2005). Historical Discourse Analysis. In *The Handbook of Discourse Analysis* (S. 138–160). John Wiley & Sons, Ltd. <https://doi.org/10.1002/9780470753460.ch8>
- Burns, P. J. (2023, 7. Mai). LatinCy: Synthetic trained pipelines for latin NLP. <https://arxiv.org/abs/2305.04365v1>
- Chang, J., Gerrish, S., Wang, C., Boyd-Graber, J. L., & Blei, D. M. (2009). Reading Tea Leaves: How Humans Interpret Topic Models. *Advances in Neural Information Processing Systems*, 288–296. Verfügbar 20. September 2017 unter <http://papers.nips.cc/paper/3700-reading-tea-leaves-how-humans-interpret-topic-models.pdf>
- Crane, G. (1998). The Perseus Project and Beyond: How Building a Digital Library Challenges the Humanities and Technology. *D-Lib Magazine*. Verfügbar 23. Februar 2026 unter <https://www.dlib.org/dlib/january98/01crane.html>
- Ebbeler, J. (2010). Letters. In *The Oxford Handbook of Roman Studies*. Oxford University Press. <https://doi.org/10.1093/oxfordhb/9780199211524.013.0029>
- Egelhaaf-Gaiser, U. (2025). Der Brief als Forschungsfeld in der Klassischen Philologie. In E.-M. Becker, U. Egelhaaf-Gaiser & A. Fürst (Hrsg.), *Hand-*

- buch Brief: Antike* (S. 1–30). De Gruyter. <https://doi.org/doi:10.1515/9783110586800-001>
- Eickhoff, F. C., Kofler, W., & Zimmermann, B. (2016). Muße, Rekursivität und antike Briefe.: Eine Einleitung. In *Muße und Rekursivität in der antiken Briefliteratur: Mit einem Ausblick in andere Gattungen* (S. 1–12). Mohr Siebeck GmbH & Co. KG. Verfügbar 1. März 2026 unter <http://www.jstor.org/stable/j.ctv9b2x8x.3>
- Foucault, M. (1991). *Die Ordnung des Diskurses* [(orig. 1971)]. Fischer Taschenbuch.
- Giebel, M. (n. d.). “Ein großes Buch, ein großes Übel” - Bücher und Bibliotheken in der Antike. Verfügbar 22. März 2026 unter https://www.fachdidaktik.klassphil.uni-muenchen.de/forschung/didaktik_waiblinger/marion_giebel/buecher_u_bib.pdf
- Grootendorst, M. (2022). BERTopic: Neural Topic Modeling with a Class-based TF-IDF Procedure. *arXiv preprint arXiv:2203.05794*.
- Hoskin, M. J. J. (2025). Letters and the Study of History. In E.-M. Becker, U. Egelhaaf-Gaiser & A. Fürst (Hrsg.), *Handbuch Brief: Antike* (S. 88–110). De Gruyter. Verfügbar 8. Februar 2026 unter <https://www.degruyterbrill.com/document/doi/10.1515/9783110586800-004/html>
- Jakobson, R. (2010, Dezember). Linguistics and Poetics. In *Volume III Poetry of Grammar and Grammar of Poetry* (S. 18–51). De Gruyter Mouton. Verfügbar 1. März 2026 unter https://www.degruyterbrill.com/document/doi/10.1515/9783110802122.18/html?srsltid=AfmB0orQcebnNeI_zufU00Jz9WmeYuH4TEY_Ct-9F_Ee2X0hW0oUBeED
- Johnson, K. P., Burns, P. J., Stewart, J., Cook, T., Besnier, C., & Mattingly, W. J. B. (2021). The Classical Language Toolkit: An NLP Framework for Pre-Modern Languages. *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: System Demonstrations*, 20–29. <https://doi.org/10.18653/v1/2021.acl-demo.3>
- Jurafsky, D., & Martin, J. H. (2019). *Speech and Language Processing (3rd draft ed.)* Stanford University.
- Mann, W. C., & Thompson, S. A. (1988). Rhetorical Structure Theory: Toward a Functional Theory of Text Organization. *Text - Interdisciplinary Journal for the Study of Discourse*, 8(3), 243–281. <https://doi.org/10.1515/text.1.1988.8.3.243>
- Manuwald, G. (2014). *Cicero*. Bloomsbury Publishing.
- Manuwald, G. (2025). Die Briefe Ciceros. In E.-M. Becker, U. Egelhaaf-Gaiser & A. Fürst (Hrsg.), *Handbuch Brief: Antike* (S. 796–803). De Gruyter. Verfügbar 8. Februar 2026 unter <https://www.degruyterbrill.com/document/doi/10.1515/9783110586800-065/html>
- Mommsen, T. (1856). *Römische Geschichte* (1. Aufl., Bd. 3). Weidmann.
- Moretti, F. (2013). *Distant Reading*. Verso Books.
- Müller, W. G. (1980). Der Brief als Spiegel der Seele. Zur Geschichte eines Topos der Epistolartheorie von der Antike bis zu Samuel Richardson. *Antike und Abendland*, 26(1), 138–157. <https://doi.org/10.1515/9783110241389.138>

- Nusch, C. J., Del Rio Riande, G., Cagnina, L. C., Errecalde, M. L., & Antonelli, L. (2025). Initial Explorations for Document Clustering Tasks in Latin Elegiac Poets. In V. Agredo-Delgado, P. H. Ruiz & C. A. Meneses Escobar (Hrsg.), *Collaboration in Knowledge Discovery and Decision Making* (S. 120–134, Bd. 2369). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-91690-8_10
- Řehůřek, R., & Sojka, P. (2010). Software Framework for Topic Modelling with Large Corpora. *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*, 45–50. <http://is.muni.cz/publication/884893/en>
- Rühl, M. (2018). *Ciceros Korrespondenz als Medium literarischen und gesellschaftlichen Handelns*. Brill. <https://doi.org/10.1163/9789004383159>
- Rühl, M. (2025). Kommunikationswissenschaft und Konversationsanalyse. In E.-M. Becker, U. Egelhaaf-Gaiser & A. Fürst (Hrsg.), *Handbuch Brief: Antike* (S. 327–336). De Gruyter. Verfügbar 8. Februar 2026 unter <https://www.degruyterbrill.com/document/doi/10.1515/9783110586800-020/html>
- Schröder, V. B.-J. (2004). Ciceros Briefe als Briefe. *Acta Classica*, XL-XLI.
- Schwitter, R. (2017). Der tröstende Freund. Epistolare Praxis und kommunikative Verhaltensweise in Cicero's Epistulae ad familiares. *Ciceroniana on line*, 1(2), 369–394.
- Seibert, S. (2025). Brieftheorie. In E.-M. Becker, U. Egelhaaf-Gaiser & A. Fürst (Hrsg.), *Handbuch Brief: Antike* (S. 223–237). De Gruyter. Verfügbar 8. Februar 2026 unter <https://www.degruyterbrill.com/document/doi/10.1515/9783110586800-012/html>
- Shiffrin, D. (2010). Discourse. In R. W. Fasold & J. Connor-Linton (Hrsg.), *Introduction to Language and Linguistics* (S. 169–204). Cambridge University Presse.
- Smith, D. A., Rydberg-Cox, J. A., & Crane, G. R. (2000). The Perseus Project: a Digital Library for the Humanities. *Literary and Linguistic Computing*, 15, 15–25. <https://api.semanticscholar.org/CorpusID:61092357>
- Soffiantini, L. (2024). Latin Funerary Language: Formulae and Abbreviations of Ancient Inscriptions. Verfügbar 22. März 2026 unter https://www.academia.edu/119915824/Epigraphic_formulaic_language_and_the_use_of_the_dispersion_measure_TF_IDF
- Viertel, A. (1879). *Die Wiederauffindung von Ciceros Briefen durch Petrarca: eine philologisch-kritische Untersuchung*. Hartung.
- Walter, A. E. (2014). *Späthumanismus und Konfessionspolitik: Die europäische Gelehrtenrepublik um 1600 im Spiegel der Korrespondenzen Georg Michael Lingelsheims* (Bd. 95). Walter de Gruyter GmbH & Co KG.
- White, P. (2010). *Cicero in Letters: Epistolary Relations of the Late Republic*. Oxford University Press.

